ArcadeHaven - DFDs

Owner: Diogo Sousa
Reviewer: Diogo Sousa & Diogo Pereira
Contributors: Diogo Pereira
Date Generated: Sun Apr 19 2026

Executive Summary

High level system description

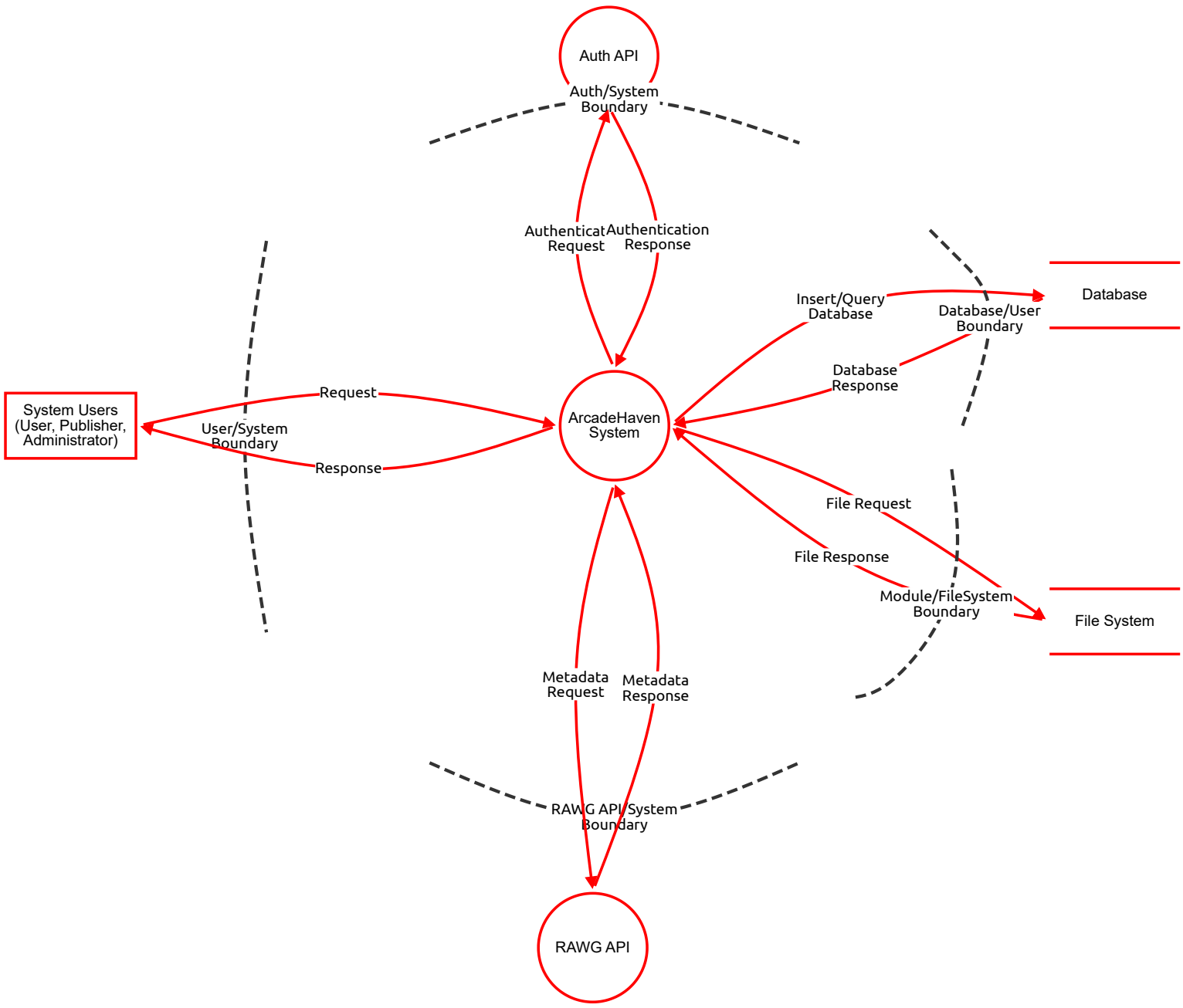
Dataflow diagrams of the ArcadeHaven system

Summary

Total Threats	308
Total Mitigated	0
Total Open	308
Open / Critical Severity	65
Open / High Severity	126
Open / Medium Severity	113
Open / Low Severity	4

DFD - Level 0

ArcadeHaven DFD - Level 0



DFD - Level 0

System Users (User, Publisher, Administrator) (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
72	User identity impersonation at system boundary	Spoofing	Critical	Open		Attacker uses stolen credentials or a forged session to interact with the system as a different user, bypassing the User/System boundary.	Keycloak MFA; strong password policies; short-lived JWTs; session binding to IP/device fingerprint.
73	Denial of system-level actions	Repudiation	High	Open		An Admin denies having performed bulk user deletions or a Buyer denies a purchase. No cross-cutting audit log exists at the system boundary.	Centralised, immutable audit log capturing all authenticated actions crossing the User/System boundary; non-repudiable receipts for purchases.

ArcadeHaven System (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
8	Request body manipulation / mass assignment	Tampering	Medium	Open		Buyer sends a POST /orders with a modified price field, paying less for a game if the API trusts client-submitted prices instead of looking up the canonical price from the DB.	Never trust client-submitted prices; resolve price server-side from the Games datastore; use strict DTO validation with @Valid; disable mass assignment via explicit @JsonProperty mapping.
9	JWT not validated on every route	Spoofing	Critical	Open		Attacker crafts a request with an unsigned or expired token to a Spring endpoint that lacks @PreAuthorize; gains access as any user.	Global JWT validation filter in Spring Security; no endpoint bypasses; token expiry enforced; alg:none attack prevention (reject tokens with no algorithm).
10	Insufficient request logging	Repudiation	High	Open		An admin deletes a user account. No structured log exists with the admin's identity, timestamp, and target. They later deny the action.	Structured access log for all mutating operations (POST/PUT/PATCH/DELETE) including authenticated user ID, resource path, body hash, and timestamp; send logs to a tamper-evident SIEM.
12	Verbose error messages expose internals	Information disclosure	Critical	Open		A 500 error leaks a stack trace containing table names, ORM query structure, or DB credentials to the caller.	Global exception handler returning generic error responses; suppress stack traces in production; use structured error codes; never include DB exception messages in HTTP responses.
13	No rate limiting on public endpoints	Denial of service	Medium	Open		Attacker floods POST /orders or GET /games with thousands of requests/second, exhausting the Spring Boot thread pool and making the API unavailable to legitimate users.	API gateway or Spring rate-limiting filter (e.g. Bucket4j); limit per IP and per authenticated user; return 429 with Retry-After; circuit breaker pattern for downstream DB calls.
14	Missing role checks on admin routes	Elevation of privilege	Critical	Open		A Buyer calls DELETE /admin/games/{id} and it succeeds because the endpoint validates only authentication (valid JWT) but not the required ADMIN role in the Keycloak token claims.	Role-based access control enforced at the method level via @PreAuthorize("hasRole('ADMIN')"); integration tests for every protected endpoint verifying that Buyer/Publisher tokens are rejected; automated security scanning in CI pipeline with SonarQube.

Auth API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
74	Rogue Auth API impersonation	Spoofing	High	Open		A misconfigured issuer URL (iss claim) points ArcadeHaven to a malicious OIDC server that issues tokens for any identity. ArcadeHaven trusts them because the signature is valid for that issuer.	Hardcode the trusted issuer URL in Spring Security config; validate iss, aud, and exp on every token; never accept tokens from unknown issuers.
75	Auth events not attributed to ArcadeHaven context	Repudiation	High	Open		Keycloak logs authentication events but they are not correlated with ArcadeHaven application-level logs, making it impossible to trace a Keycloak login to a specific ArcadeHaven order.	Propagate Keycloak session ID (sid claim) through all ArcadeHaven audit log entries to enable cross-system correlation.

RAWG API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
76	Spoofed RAWG endpoint (DNS/MitM)	Spoofing	High	Open		DNS poisoning redirects rawg.io calls to a malicious server returning crafted game metadata containing malicious URLs or scripts.	Pin RAWG base URL; enforce TLS certificate validation; treat all RAWG data as untrusted input regardless of source.
77	No record of what external data was consumed	Repudiation	Medium	Open		A bad game description sourced from RAWG causes a compliance incident. No log exists of what version of the RAWG response was ingested at what time.	Log raw RAWG responses with timestamp and request hash before processing; store a digest of consumed metadata in the Game record.

Database (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
78	Unauthorised direct DB writes	Tampering	High	Open		A developer with production DB access modifies order prices or activation keys directly in psql, bypassing all application business logic and validation.	Restrict direct DB access to break-glass procedures only; all production changes via application layer; DB user has minimal privileges (no DDL in production).
79	No DB-level change attribution	Repudiation	High	Open		A row in the users table is modified; pgaudit is disabled so there is no record of which DB user made the change or when.	Enable pgaudit; separate application DB user from migration/admin user; all schema changes via Flyway with versioned scripts committed to source control.
80	Unencrypted sensitive data at rest	Information disclosure	High	Open		PostgreSQL data directory on an unencrypted volume. A snapshot or disk image leaks all PII, activation keys, and order history.	Encrypted volumes (OS or cloud-level); application-layer encryption for activation keys; bcrypt for passwords (via Keycloak).
81	DB overload from unbounded queries	Denial of service	Medium	Open		A full-table scan on the games table with no index blocks all other DB operations for minutes.	Indices on all filter/sort columns; query timeouts in HikariCP; read replicas for heavy queries; connection pool limits.

File System (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
82	Arbitrary file write / path traversal	Tampering	High	Open		Client-supplied filename ../../etc/cron.d/shell writes a malicious cron entry outside the upload directory.	UUID filenames generated server-side; canonical path validation before any write; application runs as low-privilege OS user.
83	No file access/modification log	Repudiation	High	Open		An invoice PDF is deleted; no OS-level or application-level log records who triggered the deletion.	Application-layer file operation logging; OS auditd for sensitive directories; WORM storage for invoices.
84	Files served without auth check	Information disclosure	Medium	Open		Invoices and activation key files in a web-served directory are accessible by guessing UUID filenames.	Store sensitive files outside the web root; serve via authenticated endpoints only; time-limited signed URLs for downloads.
85	Disk exhaustion via upload abuse	Denial of service	Medium	Open		Attacker uploads hundreds of large files using a stolen Publisher token, filling the disk and blocking invoice generation.	Upload quotas per user; disk usage alerts; separate storage partition for uploads vs invoices; object storage (S3-compatible) preferred.

Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
86	MitM — request body tampering	Tampering	High	Open		HTTP (non-TLS) order request intercepted and price field modified before reaching the server.	TLS 1.2+ enforced; HSTS header; HTTP listener disabled in production.
87	Sensitive data in cleartext responses	Information disclosure	High	Open		Activation keys and PII returned over unencrypted HTTP, readable by any network observer.	TLS everywhere; Cache-Control: no-store on sensitive endpoints; no sensitive data in URLs.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
88	HTTP flood / slow-loris	Denial of service	Medium	Open		Thousands of slow connections exhaust Spring Boot thread pool, blocking legitimate users.	Nginx reverse proxy with connection and rate limits; WAF; request timeout configuration.

Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
86	MitM — request body tampering	Tampering	High	Open		HTTP (non-TLS) order request intercepted and price field modified before reaching the server.	TLS 1.2+ enforced; HSTS header; HTTP listener disabled in production.
87	Sensitive data in cleartext responses	Information disclosure	High	Open		Activation keys and PII returned over unencrypted HTTP, readable by any network observer.	TLS everywhere; Cache-Control: no-store on sensitive endpoints; no sensitive data in URLs.
88	HTTP flood / slow-loris	Denial of service	Medium	Open		Thousands of slow connections exhaust Spring Boot thread pool, blocking legitimate users.	Nginx reverse proxy with connection and rate limits; WAF; request timeout configuration.

Authentication Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
89	JWT payload tampering in transit	Tampering	High	Open		Weak HS256 key allows attacker to re-sign a modified JWT with elevated role claim.	RS256 asymmetric signing; validate against JWKS endpoint; reject alg:none tokens.
91	Token interception / exposure	Information disclosure	High	Open		Bearer token passed in URL query parameter appears in Nginx access logs and browser history.	Tokens only in Authorization header; short expiry; purge from all logs.
92	JWKS endpoint unavailability	Denial of service	High	Open		Keycloak outage causes all JWT validation to fail, making the entire API unavailable.	Cache JWKS public key locally with TTL; Keycloak HA deployment; circuit breaker.

Authentication Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
89	JWT payload tampering in transit	Tampering	High	Open		Weak HS256 key allows attacker to re-sign a modified JWT with elevated role claim.	RS256 asymmetric signing; validate against JWKS endpoint; reject alg:none tokens.
91	Token interception / exposure	Information disclosure	High	Open		Bearer token passed in URL query parameter appears in Nginx access logs and browser history.	Tokens only in Authorization header; short expiry; purge from all logs.
92	JWKS endpoint unavailability	Denial of service	High	Open		Keycloak outage causes all JWT validation to fail, making the entire API unavailable.	Cache JWKS public key locally with TTL; Keycloak HA deployment; circuit breaker.

Insert/Query Database (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).

Number	Title	Type	Severity	Status	Score	Description	Mitigations
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
93	SQL injection via flow payload	Tampering	High	Open		Unsanitised search term interpolated into native query: SELECT * FROM games WHERE title = '${input}' — input ' OR 1=1-- returns all records.	JPA parameterised queries exclusively; no string concatenation in native queries; SonarQube SQL injection rules in CI.
94	DB credentials exposed in config / logs	Information disclosure	Critical	Open		spring.datasource.password committed to GitHub; visible in build logs and SonarQube output.	Secrets in environment variables or Vault; .gitignore rules; secret scanning in CI (truffleHog).
95	Unbounded result sets exhausting JVM heap	Denial of service	Medium	Open		SELECT * FROM games with no LIMIT loads entire catalogue into memory, causing OOM.	Mandatory Pageable on all repository list methods; max page size enforced; query timeout in HikariCP.

File Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
96	Path traversal in file read/write path	Tampering	High	Open		Filename ../invoices/other_user.pdf fed to file read operation delivers another user's invoice.	Canonical path resolution and whitelisting before any read; UUID filenames; store owner mapping in DB.
97	Unauthenticated file access	Information disclosure	High	Open		Direct URL to /uploads/invoice-uuid.pdf accessible without auth if files served statically.	Authenticated file delivery endpoint; ownership verification before streaming; no static file serving for sensitive files.
98	Concurrent large file reads exhausting I/O	Denial of service	Medium	Open		Many simultaneous invoice downloads flood disk I/O, slowing DB writes needed for order processing.	Async file streaming; separate I/O thread pool for file delivery; object storage offloads I/O from application host.

File Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
96	Path traversal in file read/write path	Tampering	High	Open		Filename ../invoices/other_user.pdf fed to file read operation delivers another user's invoice.	Canonical path resolution and whitelisting before any read; UUID filenames; store owner mapping in DB.
97	Unauthenticated file access	Information disclosure	High	Open		Direct URL to /uploads/invoice-uuid.pdf accessible without auth if files served statically.	Authenticated file delivery endpoint; ownership verification before streaming; no static file serving for sensitive files.
98	Concurrent large file reads exhausting I/O	Denial of service	Medium	Open		Many simultaneous invoice downloads flood disk I/O, slowing DB writes needed for order processing.	Async file streaming; separate I/O thread pool for file delivery; object storage offloads I/O from application host.

Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
99	Malicious content in RAWG response persisted to DB	Tampering	Medium	Open		RAWG description contains a script tag; stored unsanitised; rendered as XSS to all visiting buyers.	Sanitise all external string fields before storage (OWASP Java HTML Sanitizer); output encoding at render time; CSP header.

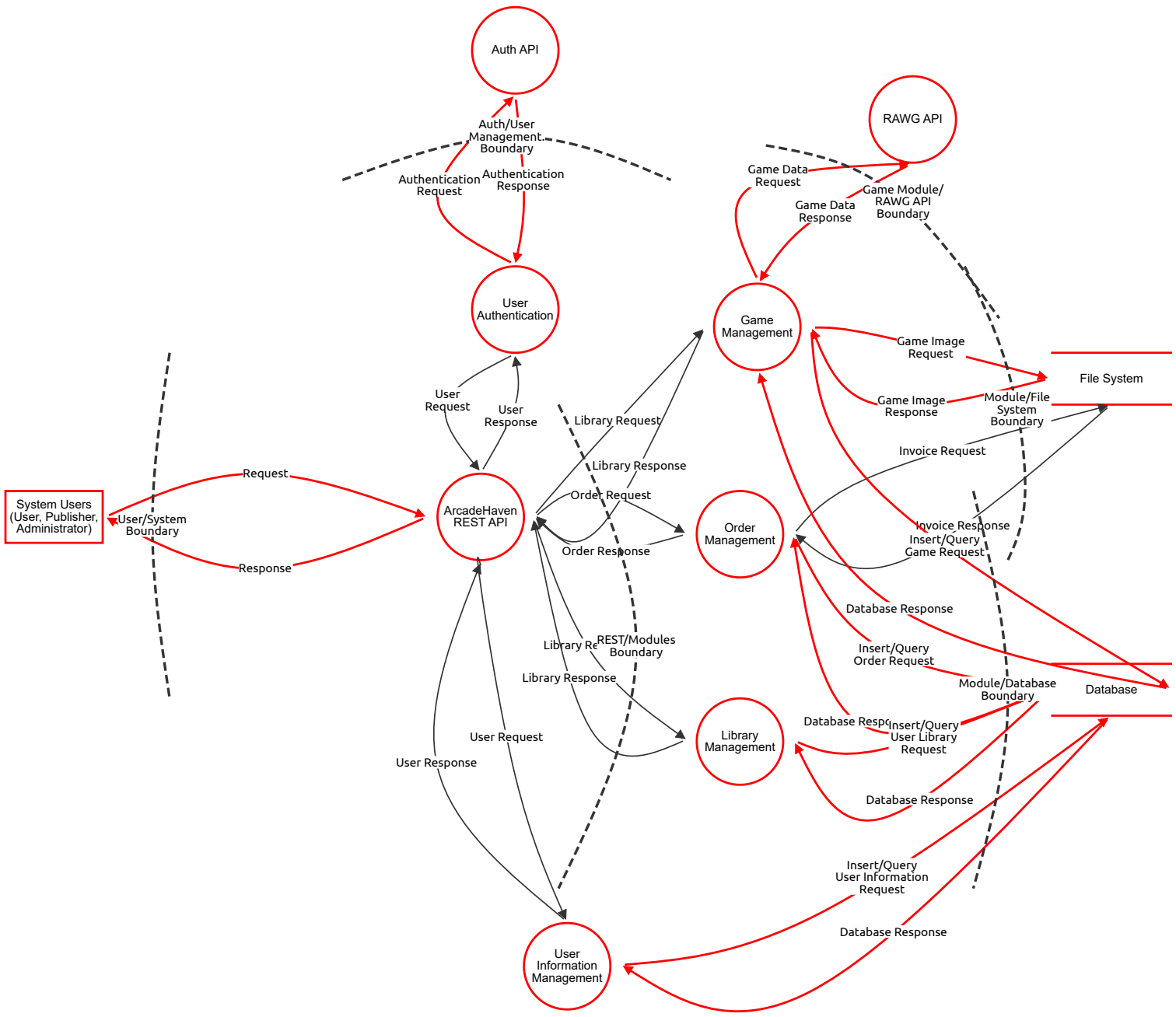
Number	Title	Type	Severity	Status	Score	Description	Mitigations
100	RAWG API key exposed	Information disclosure	Medium	Open		API key hardcoded in Java constant visible in GitHub commits and SonarQube output.	Store in environment variable / secrets manager; rotate regularly; secret scanning in CI.
101	RAWG rate limit exhaustion	Denial of service	Medium	Open		Bulk game import exhausts daily free-tier quota, breaking metadata enrichment for all users.	Client-side rate limiting; response caching with TTL; graceful degradation when RAWG unavailable; circuit breaker.

Metadata Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
99	Malicious content in RAWG response persisted to DB	Tampering	Medium	Open		RAWG description contains a script tag; stored unsanitised; rendered as XSS to all visiting buyers.	Sanitise all external string fields before storage (OWASP Java HTML Sanitizer); output encoding at render time; CSP header.
100	RAWG API key exposed	Information disclosure	Medium	Open		API key hardcoded in Java constant visible in GitHub commits and SonarQube output.	Store in environment variable / secrets manager; rotate regularly; secret scanning in CI.
101	RAWG rate limit exhaustion	Denial of service	Medium	Open		Bulk game import exhausts daily free-tier quota, breaking metadata enrichment for all users.	Client-side rate limiting; response caching with TTL; graceful degradation when RAWG unavailable; circuit breaker.

DFD - Level 1 - ArcadeHaven System

ArcadeHaven system module DFD



DFD - Level 1 - ArcadeHaven System

System Users (User, Publisher, Administrator) (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
47	User identity impersonation	Spoofing	Medium	Open		Any unauthenticated or weakly-authenticated actor can claim to be a legitimate user, publisher, or admin at the trust boundary.	An attacker reuses stolen credentials to log in as a Publisher and approve/modify game listings. An attacker forges a JWT sub claim to act as Admin.
48	Denial of performed actions	Repudiation	Medium	Open		Without audit logging, a user can deny having purchased a game, a publisher can deny having uploaded malicious content, an admin can deny having approved a game.	A Publisher uploads a game with malicious content, then claims "I never uploaded that file." No log exists to tie the upload to their identity and timestamp.

ArcadeHaven REST API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
8	Request body manipulation / mass assignment	Tampering	Medium	Open		Buyer sends a POST /orders with a modified price field, paying less for a game if the API trusts client-submitted prices instead of looking up the canonical price from the DB.	Never trust client-submitted prices; resolve price server-side from the Games datastore; use strict DTO validation with @Valid; disable mass assignment via explicit @JsonProperty mapping.
9	JWT not validated on every route	Spoofing	Critical	Open		Attacker crafts a request with an unsigned or expired token to a Spring endpoint that lacks @PreAuthorize; gains access as any user.	Global JWT validation filter in Spring Security; no endpoint bypasses; token expiry enforced; alg:none attack prevention (reject tokens with no algorithm).
10	Insufficient request logging	Repudiation	High	Open		An admin deletes a user account. No structured log exists with the admin's identity, timestamp, and target. They later deny the action.	Structured access log for all mutating operations (POST/PUT/PATCH/DELETE) including authenticated user ID, resource path, body hash, and timestamp; send logs to a tamper-evident SIEM.
12	Verbose error messages expose internals	Information disclosure	Critical	Open		A 500 error leaks a stack trace containing table names, ORM query structure, or DB credentials to the caller.	Global exception handler returning generic error responses; suppress stack traces in production; use structured error codes; never include DB exception messages in HTTP responses.
13	No rate limiting on public endpoints	Denial of service	Medium	Open		Attacker floods POST /orders or GET /games with thousands of requests/second, exhausting the Spring Boot thread pool and making the API unavailable to legitimate users.	API gateway or Spring rate-limiting filter (e.g. Bucket4j); limit per IP and per authenticated user; return 429 with Retry-After; circuit breaker pattern for downstream DB calls.
14	Missing role checks on admin routes	Elevation of privilege	Critical	Open		A Buyer calls DELETE /admin/games/{id} and it succeeds because the endpoint validates only authentication (valid JWT) but not the required ADMIN role in the Keycloak token claims.	Role-based access control enforced at the method level via @PreAuthorize("hasRole('ADMIN')"); integration tests for every protected endpoint verifying that Buyer/Publisher tokens are rejected; automated security scanning in CI pipeline with SonarQube.

User Authentication (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
15	Credential brute-force / credential stuffing	Spoofing	High	Open		Attacker tries thousands of email/password combinations from a leaked credential list against the Keycloak login endpoint to hijack accounts.	Account lockout after N failed attempts in Keycloak; CAPTCHA on login; MFA for all roles; breached-password detection; monitor for anomalous login patterns.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
16	JWT claims manipulation	Tampering	High	Open		If JWT signing key is weak or exposed, an attacker crafts a token elevating their role from BUYER to ADMIN in the realm_access.roles claim.	Use RS256 asymmetric signing in Keycloak; rotate signing keys regularly; back-end must verify signature against Keycloak's JWKS endpoint, not just parse the payload; never trust client-modified tokens.
17	No authentication event audit trail	Repudiation	High	Open		A compromised admin account performs bulk deletions. Without login/logout and token-issue audit logs, there is no proof of when the account was used or from which IP.	Enable Keycloak event logging (login, logout, token issue, failed login); forward events to a centralised log; retain logs for at least 90 days; alert on suspicious patterns (login from new geo, many failures).
18	Token leakage via insecure storage or logging	Information disclosure	High	Open		JWT access tokens are logged in Spring Boot request logs at DEBUG level. A developer with log access can impersonate any user whose token appears in the log.	Never log full Authorization headers; mask tokens in all log output; enforce HTTPS-only token transport; set SameSite=Strict and Secure on any auth cookies; use short token lifetimes.
19	Keycloak token endpoint flooded	Denial of service	High	Open		Attacker hammers POST /realms/arcadehaven/protocol/openid-connect/token with invalid credentials, locking out legitimate users or exhausting Keycloak's DB connection pool.	Rate-limit the Keycloak token endpoint at the reverse proxy / WAF layer; deploy Keycloak in HA mode with a dedicated DB; monitor token endpoint latency as a health signal.
20	Role assignment via Keycloak admin console exposure	Elevation of privilege	Critical	Open		Keycloak admin console is accessible on the public internet. An attacker exploits default credentials or an unpatched CVE to assign ADMIN role to their account.	Restrict Keycloak admin console to internal network / VPN only; change default admin credentials immediately; keep Keycloak patched; enable Keycloak audit log for all realm admin operations.

Auth API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
51	Accepting tokens from untrusted issuers	Spoofing	High	Open		Spring Security misconfigured to accept any JWT, allowing an attacker's self-issued token to authenticate as any user.	Whitelist exactly one issuer URI in Spring Security config (spring.security.oauth2.resourceserver.jwt.issuer-uri).
52	Token claims not re-validated after role change	Tampering	Medium	Open		A user's Keycloak role is downgraded by an admin, but their existing JWT (valid for 15 more minutes) still grants the old role because claims aren't re-checked against Keycloak on each request.	Implement token introspection or keep token TTL very short (≤5 min) for sensitive role changes; maintain a token revocation list or use Keycloak logout events.
53	No auth decision logging in Spring layer	Repudiation	Medium	Open		Spring Security rejects a request for insufficient privileges but logs nothing; the access attempt cannot be investigated later.	Custom AuthenticationFailureHandler and AccessDeniedHandler logging rejected attempts with user ID, resource, and timestamp.
54	Token logged at DEBUG level	Information disclosure	Medium	Open		Spring Boot DEBUG logging includes full Authorization header; a developer with log access can impersonate any logged user.	Never log Authorization headers; mask tokens in all log formatters; use structured logging with explicit field whitelists.
55	Repeated JWKS fetches on every request	Denial of service	Critical	Open		JWKS endpoint called on every API request; Keycloak slowdown cascades to full API unavailability.	Cache JWKS keys locally with configurable TTL; background refresh; circuit breaker around JWKS fetch.
56	Role extracted from wrong JWT field	Elevation of privilege	High	Open		Spring Security reads roles from realm_access.roles but attacker crafts a token with roles in a custom claim that is mistakenly also trusted, granting ADMIN	Explicitly configure which JWT claim path maps to Spring authorities; reject tokens with unexpected claim structures.

Game Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
21	Publisher identity spoofing	Spoofing	High	Open		Attacker obtains a valid Publisher JWT and submits games on behalf of a legitimate publisher, polluting their catalogue with malicious titles.	Validate JWT publisher ID against the game's owning publisher in every write operation; RBAC enforcement at method level; Keycloak MFA for Publisher accounts.
22	Malicious file upload (images/game files)	Tampering	High	Open		A Publisher uploads a .exe disguised as a game screenshot. If the server only validates Content-Type header (spoofable), the file is stored as-is and served to buyers.	Server-side file type validation using magic bytes (not just extension or Content-Type); antivirus/malware scanning pipeline on all uploads (e.g. ClamAV); store uploads outside the web root; generate a new UUID filename on save; restrict executable file types.
23	Unsigned game status changes	Repudiation	Medium	Open		Admin approves a game (PENDING → ACTIVE) but no log records which admin, when, or from which IP. Admin later denies the approval after a compliance audit.	Audit log every game state transition with actor identity, timestamp, and previous/new state; store in append-only table; require admin comment on approval/rejection.
24	RAWG metadata injection into stored fields	Information disclosure	Medium	Open		RAWG API returns a game description containing a script tag. If stored unsanitised and later rendered in HTML, it becomes a stored XSS vector disclosing session tokens of visiting users.	Sanitise all RAWG-sourced fields before storage using an HTML sanitiser (e.g. OWASP Java HTML Sanitizer); apply output encoding at the frontend; enforce Content-Security-Policy.
25	Large file upload exhausting disk/memory	Denial of service	Medium	Open		Publisher (or attacker with a stolen Publisher token) uploads a 10 GB file as a "game screenshot", consuming all available disk on the file system and blocking invoice generation.	Enforce maximum file size limits at the API gateway and in the multipart upload handler; check available disk space before write; implement per-publisher upload quota; alert on disk usage thresholds.
26	Publisher self-approving games	Elevation of privilege	High	Open		If the ADMIN role check on the game approval endpoint is missing, a Publisher can call PATCH /games/{id}/approve and set their own game to ACTIVE, bypassing admin review.	Strictly separate Publisher and Admin operations in the controller layer; approval endpoint requires ADMIN role; automated integration test verifies Publisher token returns 403 on approval endpoint.

Order Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
27	Placing orders on behalf of another user	Spoofing	High	Open		Attacker intercepts a Buyer's JWT and submits an order that charges the victim's account while delivering the activation key to the attacker's library.	Bind order's buyer_id server-side from the authenticated JWT sub claim, never from the request body; validate JWT on every order endpoint; revoke compromised tokens via Keycloak session invalidation.
28	Order price or item count manipulation	Tampering	Medium	Open		Buyer modifies the POST /orders request body to set price: 0.01 for a €59.99 game. If the server processes the client-supplied price, the item is effectively free.	Always resolve item price from the Games datastore at order creation time; never accept price in the order creation payload; recalculate totals server-side; validate order items against active game catalogue.
29	Disputing completed purchases	Repudiation	Medium	Open		Buyer claims "I never bought that game." Without a signed, timestamped order record, the system cannot prove the purchase occurred with the user's authenticated session.	Immutable order records with buyer ID, timestamp, game ID, price at purchase, and invoice PDF hash; generate and store non-repudiable receipts; consider order signing with a server-side key for high-value transactions.
30	Activation key leakage in API responses	Information disclosure	Medium	Open		GET /orders/{id} returns the activation key in plain JSON. If the API response is cached by a CDN or proxy, the key is exposed to anyone with a cached response URL.	Never cache endpoints that return activation keys (Cache-Control: no-store); serve activation keys only after re-authentication confirmation; consider one-time retrieval tokens for key delivery; encrypt keys at rest in the DB.
31	Order flooding / cart abuse	Denial of service	High	Open		Attacker creates thousands of incomplete orders per second, exhausting DB connection pool and preventing legitimate checkouts. Invoice PDF generation (CPU-intensive) can be weaponised similarly.	Rate-limit order creation per authenticated user; implement async invoice generation (message queue); limit pending/abandoned orders per user; add circuit breaker around PDF generation service.
32	Accessing another user's order history	Elevation of privilege	Medium	Open		Buyer calls GET /orders?userId=2 and retrieves another user's full purchase history and activation keys because the endpoint filters only by the query param, not the JWT sub.	Filter all order queries by the authenticated user's ID from the JWT, ignoring any userId parameter in the request; add automated IDOR (Insecure Direct Object Reference) tests to the CI pipeline.

Library Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
33	Accessing another user's library	Spoofing	Medium	Open		Attacker crafts GET /library/{userId} where userId belongs to another Buyer, reading their full game collection and activation keys.	Library access enforced to authenticated user's own ID from JWT; no userId parameter accepted from client for personal library operations; strict IDOR prevention.
34	Injecting games into another user's library	Tampering	Medium	Open		An attacker with ADMIN-level access (or exploiting missing auth) calls POST /library/{victimUserId}/entries to grant games without purchase, bypassing the order flow.	Library write operations gated to Order Management completion events only (event-driven entry creation); no direct external API to add library entries; admin library modifications require extra audit trail.
35	Untracked profile changes	Repudiation	Medium	Open		A user changes their email address (used for invoice delivery) then disputes a charge, claiming "That's not my email." No audit log shows they changed it themselves.	Log all profile mutations (email, password, display name) with timestamp and actor ID; require re-authentication (current password) for sensitive profile changes; send notification email to old address on change.
36	PII exposure in API responses	Information disclosure	Medium	Open		GET /users/{id} returns full user object including hashed password, internal role flags, and email for all callers, not just the owner, due to missing @JsonIgnore on sensitive fields.	Use dedicated response DTOs that exclude internal fields; apply @JsonIgnore to sensitive fields; ensure GET /users/{id} returns only data the authenticated caller is entitled to; never return password hashes.
37	Library query returning unbounded results	Denial of service	Medium	Open		A user with 10,000 library entries calls GET /library without pagination. The ORM loads all records into memory, causing OOM error or extreme latency.	Mandatory pagination on all list endpoints (max page size enforced server-side); use @PageableDefault with a maximum page size limit; lazy-load relationships in JPA/Hibernate.
38	Horizontal privilege escalation via profile edit	Elevation of privilege	High	Open		Buyer calls PATCH /users/{adminUserId}/profile and, if the endpoint only checks for authentication rather than ownership, modifies an admin's email or display name.	All user profile updates must verify that the authenticated user's ID matches the target resource owner; admins managing other accounts require explicit ADMIN role check; prevent users from modifying their own roles.

User Information Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
33	Accessing another user's library	Spoofing	Medium	Open		Attacker crafts GET /library/{userId} where userId belongs to another Buyer, reading their full game collection and activation keys.	Library access enforced to authenticated user's own ID from JWT; no userId parameter accepted from client for personal library operations; strict IDOR prevention.
34	Injecting games into another user's library	Tampering	Medium	Open		An attacker with ADMIN-level access (or exploiting missing auth) calls POST /library/{victimUserId}/entries to grant games without purchase, bypassing the order flow.	Library write operations gated to Order Management completion events only (event-driven entry creation); no direct external API to add library entries; admin library modifications require extra audit trail.
35	Untracked profile changes	Repudiation	Medium	Open		A user changes their email address (used for invoice delivery) then disputes a charge, claiming "That's not my email." No audit log shows they changed it themselves.	Log all profile mutations (email, password, display name) with timestamp and actor ID; require re-authentication (current password) for sensitive profile changes; send notification email to old address on change.
36	PII exposure in API responses	Information disclosure	Medium	Open		GET /users/{id} returns full user object including hashed password, internal role flags, and email for all callers, not just the owner, due to missing @JsonIgnore on sensitive fields.	Use dedicated response DTOs that exclude internal fields; apply @JsonIgnore to sensitive fields; ensure GET /users/{id} returns only data the authenticated caller is entitled to; never return password hashes.
37	Library query returning unbounded results	Denial of service	Medium	Open		A user with 10,000 library entries calls GET /library without pagination. The ORM loads all records into memory, causing OOM error or extreme latency.	Mandatory pagination on all list endpoints (max page size enforced server-side); use @PageableDefault with a maximum page size limit; lazy-load relationships in JPA/Hibernate.
38	Horizontal privilege escalation via profile edit	Elevation of privilege	High	Open		Buyer calls PATCH /users/{adminUserId}/profile and, if the endpoint only checks for authentication rather than ownership, modifies an admin's email or display name.	All user profile updates must verify that the authenticated user's ID matches the target resource owner; admins managing other accounts require explicit ADMIN role check; prevent users from modifying their own roles.

RAWG API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
49	Rogue/spoofed RAWG API endpoint	Spoofing	High	Open		The system calls an external REST API. DNS poisoning or a supply-chain compromise could redirect calls to a malicious server returning crafted metadata.	Attacker performs DNS cache poisoning so rawg.io resolves to their server, which returns forged game metadata (e.g. injecting malicious URLs as game image links) that gets stored in the database.
50	No accountability for bad external data	Repudiation	Medium	Open		There is no mechanism to prove which version of RAWG data was consumed at a given time, making it impossible to attribute data-quality incidents.	RAWG returns incorrect age-rating metadata; a minor accesses an adult game. There is no record of what was returned at the time of ingestion, making root-cause analysis impossible.

File System (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
43	Path traversal / arbitrary file write	Tampering	High	Open		A Publisher submits a filename of ../../etc/cron.d/backdoor as the game image filename. If the application uses the client-supplied filename directly, it writes the file outside the intended upload directory.	Always generate a server-side UUID filename; never use client-supplied filenames in file system paths; resolve the canonical path and verify it is within the intended base directory before writing; run the application with minimal OS file-system permissions.
44	No file access/modification audit trail	Repudiation	High	Open		An invoice PDF file is modified or deleted. Without file-system access logging, it is impossible to determine who accessed or changed it, violating financial audit requirements.	Log all file read/write/delete operations at the application layer with actor ID and timestamp; use OS-level file integrity monitoring (e.g. auditd on Linux) for the invoices directory; consider WORM (write-once-read-many) storage for invoice PDFs.
45	Direct file access bypassing application auth	Information disclosure	High	Open		Invoice PDFs and activation key files are stored in a directory served directly by the web server (e.g. Nginx static file serving). Guessing the file path allows unauthenticated download of any invoice.	Store sensitive files (invoices, activation key files) outside the web-serving root; serve them only via authenticated application endpoints that verify the requester owns the resource; generate time-limited signed download URLs for file delivery.
46	Disk exhaustion via upload abuse	Denial of service	Medium	Open		An attacker with a valid Publisher token uploads hundreds of large image files in rapid succession, filling the disk partition and preventing invoice PDF generation, which requires disk writes to complete orders.	Enforce per-user and global upload size quotas; monitor disk usage with automated alerts at 70%/85%/95% thresholds; store uploads on a separate partition or object storage (e.g. S3) isolated from application and invoice storage; implement async file processing with backpressure.

Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
60	MitM — request/response body modification	Tampering	High	Open		An attacker on the network path intercepts an HTTP (non-TLS) order creation request and modifies the game_id or quantity before it reaches the server.	Enforce HTTPS (TLS 1.2+) for all traffic; HTTP Strict Transport Security (HSTS) header; disable HTTP listener entirely in production; use certificate pinning in the frontend if applicable.
61	Sensitive data in HTTP responses over unencrypted channel	Information disclosure	Medium	Open		Activation keys, invoice data, and PII returned in API responses are transmitted in cleartext over HTTP, readable by any network observer.	TLS everywhere; mark all sensitive response fields as non-cacheable (Cache-Control: no-store, no-cache); use secure, HttpOnly, SameSite cookies for any session identifiers; avoid putting sensitive data in URL query parameters (logged by proxies and servers).
62	HTTP flood / slow-loris attack	Denial of service	High	Open		Attacker opens thousands of slow HTTP connections to the Spring Boot server, holding threads open without completing requests, exhausting the server thread pool and blocking legitimate traffic.	Deploy behind a reverse proxy (Nginx) that handles connection timeouts; set maximum request body size and read timeout; use a WAF to detect and block HTTP flood patterns; Nginx's limit_conn and limit_req modules; configure Spring Boot's server.connection-timeout.

Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
60	MitM — request/response body modification	Tampering	High	Open		An attacker on the network path intercepts an HTTP (non-TLS) order creation request and modifies the game_id or quantity before it reaches the server.	Enforce HTTPS (TLS 1.2+) for all traffic; HTTP Strict Transport Security (HSTS) header; disable HTTP listener entirely in production; use certificate pinning in the frontend if applicable.
61	Sensitive data in HTTP responses over unencrypted channel	Information disclosure	Medium	Open		Activation keys, invoice data, and PII returned in API responses are transmitted in cleartext over HTTP, readable by any network observer.	TLS everywhere; mark all sensitive response fields as non-cacheable (Cache-Control: no-store, no-cache); use secure, HttpOnly, SameSite cookies for any session identifiers; avoid putting sensitive data in URL query parameters (logged by proxies and servers).
62	HTTP flood / slow-loris attack	Denial of service	High	Open		Attacker opens thousands of slow HTTP connections to the Spring Boot server, holding threads open without completing requests, exhausting the server thread pool and blocking legitimate traffic.	Deploy behind a reverse proxy (Nginx) that handles connection timeouts; set maximum request body size and read timeout; use a WAF to detect and block HTTP flood patterns; Nginx's limit_conn and limit_req modules; configure Spring Boot's server.connection-timeout.

Authentication Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
57	Token header/payload tampering	Tampering	Medium	Open		Attacker intercepts a JWT, modifies the role claim in the payload from BUYER to ADMIN, and re-signs with a known weak secret (HS256 with secret "secret").	Use asymmetric signing (RS256) from Keycloak; backend validates signature against JWKS endpoint; reject tokens signed with HS256 or weak keys; validate all standard claims (iss, aud, exp, nbf).
58	Token interception / exposure in logs	Information disclosure	High	Open		Bearer token is included in a GET request URL as a query parameter (?token=...) which is logged by Nginx access logs, a CDN, and browser history — exposing a long-lived token.	Always transmit JWTs in the Authorization header, never in URLs; set short expiry on access tokens (≤15 min); use refresh token rotation; purge tokens from all log pipelines; monitor for token reuse from unexpected IPs.
59	Token validation overhead / JWKS endpoint unavailability	Denial of service	Medium	Open		If Spring Security fetches the JWKS endpoint on every request to validate token signatures, a Keycloak outage causes all API requests to fail. Alternatively, an attacker floods the JWKS endpoint.	Cache the JWKS public key locally with a reasonable TTL (e.g. 1 hour); implement offline token validation fallback; deploy Keycloak in HA configuration behind a load balancer; health-check the JWKS endpoint in the application startup; use circuit breaker for JWKS fetching.

Authentication Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
57	Token header/payload tampering	Tampering	Medium	Open		Attacker intercepts a JWT, modifies the role claim in the payload from BUYER to ADMIN, and re-signs with a known weak secret (HS256 with secret "secret").	Use asymmetric signing (RS256) from Keycloak; backend validates signature against JWKS endpoint; reject tokens signed with HS256 or weak keys; validate all standard claims (iss, aud, exp, nbf).
58	Token interception / exposure in logs	Information disclosure	High	Open		Bearer token is included in a GET request URL as a query parameter (?token=...) which is logged by Nginx access logs, a CDN, and browser history — exposing a long-lived token.	Always transmit JWTs in the Authorization header, never in URLs; set short expiry on access tokens (≤15 min); use refresh token rotation; purge tokens from all log pipelines; monitor for token reuse from unexpected IPs.
59	Token validation overhead / JWKS endpoint unavailability	Denial of service	Medium	Open		If Spring Security fetches the JWKS endpoint on every request to validate token signatures, a Keycloak outage causes all API requests to fail. Alternatively, an attacker floods the JWKS endpoint.	Cache the JWKS public key locally with a reasonable TTL (e.g. 1 hour); implement offline token validation fallback; deploy Keycloak in HA configuration behind a load balancer; health-check the JWKS endpoint in the application startup; use circuit breaker for JWKS fetching.

User Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

User Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Order Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Order Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

User Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

User Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Data Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Data Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Image Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Game Image Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Invoice Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Invoice Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Insert/Query Game Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Insert/Query Order Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Insert/Query User Library Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Insert/Query User Information Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

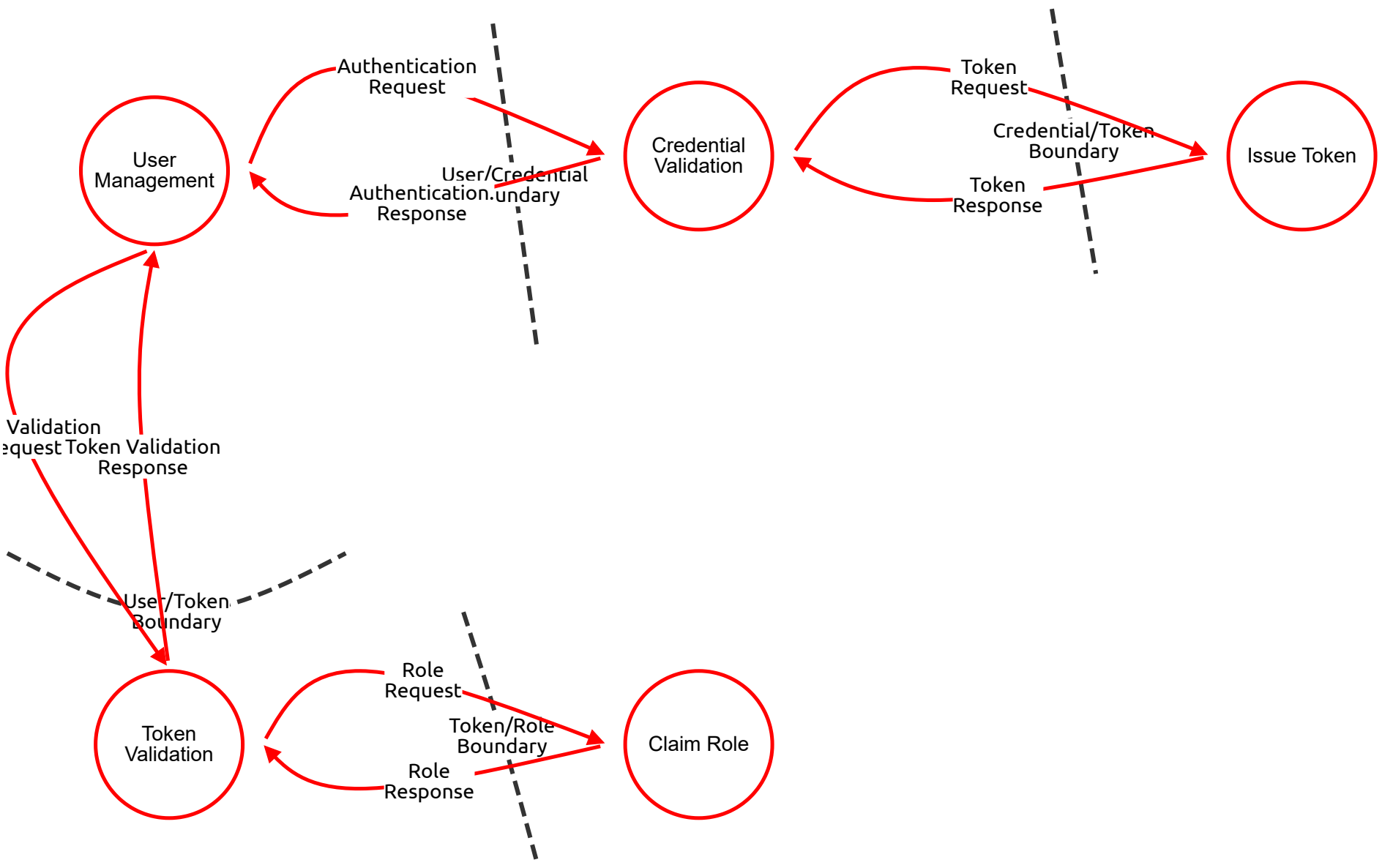
Database (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
39	SQL injection / ORM injection	Tampering	Critical	Open		A malformed game search query bypasses the JPA parameterisation (e.g. via a native query built with string concatenation) and executes DROP TABLE orders; in the database.	Use only parameterised queries and JPA named parameters; never use string concatenation in native queries; apply principle of least privilege to the DB user (no DROP/DDL rights in application account); run Flyway migrations with a separate privileged user.
40	No DB-level audit trail	Repudiation	High	Open		A record in the orders table is modified directly (bypassing the application layer, e.g. by a DBA). Without DB-level change logging, the modification cannot be attributed or detected.	Enable PostgreSQL pgaudit extension to log all DDL and DML; configure row-level security where appropriate; use Flyway for all schema changes; restrict direct DB access to emergency procedures with full logging; consider application-level audit tables with insert-only access.
41	Unencrypted PII and activation keys at rest	Information disclosure	High	Open		The PostgreSQL data directory is stored unencrypted. A cloud storage misconfiguration or disk theft exposes all user PII, hashed passwords, and plaintext activation keys.	Enable PostgreSQL Transparent Data Encryption or use encrypted AWS RDS/EBS volumes; encrypt sensitive columns (activation keys, payment references) at the application layer using AES-256 before storing; hash passwords with bcrypt (already handled by Spring Security / Keycloak); restrict network access to DB to application subnet only.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
42	Connection pool exhaustion / long-running queries	Denial of service	High	Open		An attacker triggers a complex game search with many filters, generating a full-table-scan query that holds DB connections for minutes, starving other requests of connections from HikariCP pool.	Set query timeouts in Spring Data / JPA (spring.transaction.default-timeout); add DB indices on search columns (game title, category, publisher_id); configure HikariCP connection pool limits and timeouts; use read replicas for search/reporting queries; implement circuit breaker for DB calls.

DFD - Level 1 - Auth API

Auth API DFD



DFD - Level 1 - Auth API

User Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
103	Keycloak admin console exposed publicly	Spoofing	Critical	Open		Attacker reaches /auth/admin/arcadehaven/console from the internet and brute-forces the admin password, gaining control of the entire user realm.	Restrict admin console to internal network / VPN; change default credentials; enable MFA on Keycloak admin account; fail2ban on admin login endpoint.
104	Unauthorised user record modification	Tampering	High	Open		An attacker with Keycloak manage-users role (but not full admin) modifies another admin's email address, enabling account takeover via password reset.	Apply principle of least privilege on Keycloak service account roles; ArcadeHaven application uses only view-users scope; admin operations require dedicated admin account with MFA.
105	No Keycloak admin event logging	Repudiation	High	Open		Admin creates a new PUBLISHER user with elevated realm roles; no Keycloak admin event log entry is kept; action cannot be audited.	Enable Keycloak Admin Events logging in realm settings; forward events to SIEM; retain for 90+ days; alert on role assignment events.
106	User PII exposed via Keycloak admin API	Information disclosure	High	Open		ArcadeHaven back-end service account has manage-users scope; a vulnerability in the application allows an attacker to proxy calls to /auth/admin/users and enumerate all user emails and profile data.	Grant ArcadeHaven service account only the minimum Keycloak scopes needed (e.g. view-profile); never manage-users in the application service account; separate admin automation account.
107	Mass user creation flooding Keycloak DB	Denial of service	High	Open		Attacker exploits an open registration endpoint to create millions of accounts, filling Keycloak's internal DB and slowing authentication for all users.	CAPTCHA and rate limiting on user registration; email verification before account activation; monitor registration rate; Keycloak registration event alerts.
108	Service account with excessive Keycloak roles	Elevation of privilege	High	Open		ArcadeHaven's Spring Boot service account is granted realm-admin role in Keycloak. If the application is compromised, the attacker inherits full realm control.	Service account uses only the minimum scopes needed; never realm-admin; rotate service account credentials; audit service account permissions regularly.

Credential Validation (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
109	Credential stuffing / brute-force	Spoofing	Critical	Open		Attacker uses leaked credential list from another breach to log in to ArcadeHaven accounts at scale.	Keycloak brute-force protection (account lockout after N failures); MFA requirement; breached-password check via HaveIBeenPwned API at registration and login.
110	Password hash algorithm downgrade	Tampering	High	Open		Keycloak is misconfigured to store passwords using MD5 instead of bcrypt/Argon2; a DB leak leads to rapid hash crackin	Verify Keycloak password hashing algorithm is set to bcrypt or Argon2 in realm settings; never use MD5/SHA1 for passwords; audit Keycloak configuration as part of deployment checklist.
111	No failed-login event record	Repudiation	Medium	Open		Attacker runs credential stuffing attack for hours; no failed login events are stored, so the attack is invisible to incident response.	Enable Keycloak login event persistence; alert on N failed logins per account per hour; forward to SIEM for anomaly detection.
112	User enumeration via differential responses	Information disclosure	Medium	Open		Credential validation returns "user not found" for unknown email vs "wrong password" for known email, allowing attacker to enumerate registered users.	Keycloak returns generic "invalid credentials" for all failures by default — verify this is not overridden; constant-time comparison prevents timing-based enumeration.
113	Flood of validation requests causing Keycloak DB lock contention	Denial of service	High	Open		High-volume brute-force attack creates heavy DB write load (failed attempt counters) causing lock contention and slowing legitimate logins.	Rate-limit at reverse proxy before requests reach Keycloak; Keycloak HA with dedicated DB; failed attempt counter update can be async/eventual.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
114	Bypass of MFA via account recovery flow	Elevation of privilege	Critical	Open		Attacker uses "forgot password" flow to reset a victim's password without triggering MFA, effectively bypassing the second factor.	Ensure account recovery requires email verification to a trusted address; do not bypass MFA via recovery flows; alert on password reset events for accounts with MFA enabled.

Issue Token (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
115	Token issued to wrong client (audience confusion)	Spoofing	Critical	Open		Token issued for Keycloak client arcadehaven-frontend is accepted by arcadehaven-backend even though the aud claim should restrict it. Attacker replays a token obtained from a different client.	Validate aud claim strictly in Spring Security; each client (frontend, backend) has its own client ID; backend resource server only accepts tokens with its own client ID as audience.
116	Weak signing key allows token forgery	Tampering	High	Open		Keycloak configured with HS256 and a short, guessable secret. Attacker brute-forces the key offline and forges tokens with arbitrary claims.	Use RS256 or ES256 (asymmetric) exclusively; minimum 2048-bit RSA keys; rotate signing keys annually or on suspected compromise; backend never holds the private key.
117	No record of token issuance events	Repudiation	High	Open		A token is issued to a compromised device; no Keycloak token issuance event is logged, making it impossible to trace when the compromise began.	Enable Keycloak token event logging; log client ID, user ID, IP, and timestamp for every issued token; retain logs for at least 30 days.
118	Excessive claims in token payload	Information disclosure	High	Open		Keycloak token mapper includes user's email, phone number, and home address in the JWT payload. Any service receiving the token can read this PII from the decoded payload.	Minimise claims in token payload to only what consuming services need (sub, roles, email); use userinfo endpoint for additional attributes rather than embedding in the token.
119	Token endpoint flooded	Denial of service	High	Open		Attacker hammers POST /token with invalid grant_type values; each request hits the DB to look up the client configuration, exhausting DB connections.	Rate-limit the token endpoint at the reverse proxy / WAF; CAPTCHA for public clients; Keycloak cluster with dedicated DB pool.
120	Long-lived access tokens granting stale privileges	Elevation of privilege	High	Open		User's role is revoked by admin but their 8-hour access token remains valid, granting the revoked privilege until expiry.	Short access token TTL (5–15 min); implement token revocation list or Keycloak logout events pushed to Spring Security; use refresh token rotation so revocation is effective at next refresh.

Token Validation (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
121	Algorithm confusion attack	Spoofing	High	Open		Attacker submits JWT with alg: none in the header. A naive validator skips signature verification and accepts any payload.	Explicitly whitelist allowed algorithms (RS256 only); reject tokens with alg:none or unexpected algorithms; use well-maintained JOSE library (e.g. Nimbus JOSE+JWT).
122	Expired token accepted due to clock skew misconfiguration	Tampering	High	Open		Validator allows 1-hour clock skew "to be safe"; attacker replays a token expired 45 minutes ago.	Max allowed clock skew ≤ 30 seconds; synchronise all server clocks with NTP; log and alert on tokens with exp in the past beyond the skew window.
124	Validation failures not logged	Repudiation	Medium	Open		A flood of invalid tokens is sent to probe the system; no validation failure log is kept, making intrusion detection impossible.	Log all token validation failures with reason (expired, bad signature, wrong issuer), source IP, and timestamp; alert on anomalous failure rates.
125	Token validation errors reveal internal config	Information disclosure	Critical	Open		Validation error message "Expected issuer: https://keycloak.internal:8080/realms/arcadehaven" exposes internal Keycloak hostname and realm name to the caller.	Return generic 401 Unauthorized with no detail on validation failure reason; log details internally only.
126	Validation process overloaded by malformed tokens	Denial of service	High	Open		Attacker sends thousands of malformed JWTs requiring complex parsing; CPU exhaustion from repeated crypto operations.	Pre-validate token structure (header + payload + signature format) before attempting cryptographic verification; rate-limit per IP before validation; circuit breaker.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
127	JWKS key confusion — accepting keys from attacker-controlled URL	Elevation of privilege	High	Open		JWT header contains jku (JWK Set URL) pointing to attacker's server with their own public key; validator fetches and trusts it, accepting attacker's forged token	Never follow jku or x5u headers in tokens; only use pre-configured JWKS endpoint; ignore any key-URL hints in token headers.

Claim Role (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
128	Role claim from wrong JWT path trusted	Spoofing	High	Open		Spring Security reads roles from a custom claim path the attacker controls rather than the canonical realm_access.roles path, allowing role injection.	Explicitly configure the single authoritative JWT claim path for role extraction; discard all other potential role claims.
129	Role claim modified post-issuance	Tampering	High	Open		If the application caches decoded role claims beyond token expiry, an attacker who obtains a cached session can retain a revoked role.	Re-extract roles from the JWT on every request (no caching of role claims beyond the request scope); rely on Spring Security's SecurityContext which is per-request.
130	Role-based access decisions not logged	Repudiation	Critical	Open		An ADMIN-role operation is performed; the audit log records the user ID but not the role that authorised the operation, making privilege reviews impossible.	Include the effective role(s) from the JWT in every audit log entry for privileged operations.
131	Role information leaked in API error responses	Information disclosure	High	Open		403 response body states "requires role ADMIN"; an attacker learns the exact role name needed, aiding privilege escalation attempts.	Return generic 403 Forbidden with no role-specific detail; internal logs contain the detailed reason.
132	Complex role hierarchy causing authorisation latency	Denial of service	Medium	Open		A deeply nested Keycloak composite role structure requires many lookups to resolve effective permissions; under load, authorisation latency spikes and timeouts occur.	Keep Keycloak role hierarchy flat (3 simple roles: ADMIN, PUBLISHER, BUYER); avoid composite roles; role resolution should be O(1) from JWT claims.
133	Default role assigned to all new users is too permissive	Elevation of privilege	Medium	Open		Keycloak realm's default role is PUBLISHER; every newly registered user automatically has Publisher capabilities without explicit assignment.	Set default realm role to BUYER (least privilege); PUBLISHER and ADMIN roles must be explicitly assigned by an admin; verify default role assignment in Keycloak realm configuration.

Authentication Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
134	Credential interception and modification in transit	Tampering	Critical	Open		If Keycloak sub-components communicate over an unencrypted internal network (e.g. within a Docker bridge network without TLS), an attacker who has compromised any host on that network can intercept the credential flow and substitute a different username, authenticating as a victim without knowing their password.	Enforce TLS on all Keycloak internal component communication; use Docker network isolation so only the application container can reach Keycloak; mutual TLS for any service-to-service authentication calls; deploy Keycloak behind a private network segment not reachable from the public internet.
135	Plaintext credentials exposed in Keycloak request logs	Information disclosure	Critical	Open		Keycloak DEBUG or TRACE logging is enabled in a production environment. The full POST /token request body — including username and password fields — is written to the application log. A developer, log aggregator, or SIEM system now holds plaintext credentials for every user who has logged in.	Strictly disable DEBUG/TRACE logging in production Keycloak; configure log sanitisation to mask password and client_secret fields at the log formatter level; audit logging configuration as part of every deployment; use structured logging with an explicit field allowlist that excludes credential fields.
136	Authentication request flood causing DB lock contention	Denial of service	High	Open		An attacker sends thousands of authentication requests per second targeting the credential validation flow. Each request triggers a DB read (user lookup) and a bcrypt comparison (CPU-intensive). Keycloak's DB connection pool is exhausted and the bcrypt thread pool is saturated, causing login failures for all legitimate users platform-wide.	Rate-limit the authentication endpoint at the reverse proxy (Nginx limit_req) before requests reach Keycloak; CAPTCHA on the login form for public clients; Keycloak cluster with a dedicated DB; async bcrypt comparison to avoid blocking request threads; alert on authentication request rate spikes.

Authentication. Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
134	Credential interception and modification in transit	Tampering	Critical	Open		If Keycloak sub-components communicate over an unencrypted internal network (e.g. within a Docker bridge network without TLS), an attacker who has compromised any host on that network can intercept the credential flow and substitute a different username, authenticating as a victim without knowing their password.	Enforce TLS on all Keycloak internal component communication; use Docker network isolation so only the application container can reach Keycloak; mutual TLS for any service-to-service authentication calls; deploy Keycloak behind a private network segment not reachable from the public internet.
135	Plaintext credentials exposed in Keycloak request logs	Information disclosure	Critical	Open		Keycloak DEBUG or TRACE logging is enabled in a production environment. The full POST /token request body — including username and password fields — is written to the application log. A developer, log aggregator, or SIEM system now holds plaintext credentials for every user who has logged in.	Strictly disable DEBUG/TRACE logging in production Keycloak; configure log sanitisation to mask password and client_secret fields at the log formatter level; audit logging configuration as part of every deployment; use structured logging with an explicit field allowlist that excludes credential fields.
136	Authentication request flood causing DB lock contention	Denial of service	High	Open		An attacker sends thousands of authentication requests per second targeting the credential validation flow. Each request triggers a DB read (user lookup) and a bcrypt comparison (CPU-intensive). Keycloak's DB connection pool is exhausted and the bcrypt thread pool is saturated, causing login failures for all legitimate users platform-wide.	Rate-limit the authentication endpoint at the reverse proxy (Nginx limit_req) before requests reach Keycloak; CAPTCHA on the login form for public clients; Keycloak cluster with a dedicated DB; async bcrypt comparison to avoid blocking request threads; alert on authentication request rate spikes.

Token Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
137	Token payload tampered before signing — role claim injection	Tampering	Critical	Open		The Token Request flow carries the user's identity and role claims from Credential Validation to Issue Token. If this internal flow is interceptable (e.g. over an unencrypted message bus or shared memory), an attacker could inject an ADMIN role claim into the request payload before the token is signed, resulting in a legitimately signed token with fraudulent privileges.	For a standard Keycloak deployment this flow is in-process and not network-exposed — verify this is the case; if custom Keycloak SPIs or external mappers are used, ensure they communicate over secured channels; audit all custom Keycloak token mappers in the codebase for claim injection vulnerabilities.
138	Issued token intercepted before delivery to client (token theft)	Tampering	Critical	Open		The Token Response carries the signed JWT back to the client. If delivered over HTTP (no TLS), an on-path attacker captures the access token and refresh token. They can now authenticate as the victim indefinitely (until the refresh token expires or is revoked).	Enforce HTTPS for all token delivery endpoints; HSTS header; refresh token delivered only in HttpOnly + Secure + SameSite=Strict cookie; access token in response body (short TTL ≤15 min); implement refresh token rotation so any interception is detectable via reuse detection
139	Token response cached by intermediate proxy — token leakage	Information disclosure	High	Open		A misconfigured reverse proxy or CDN caches the token endpoint response (missing Cache-Control: no-store). A subsequent request from a different user receives another user's JWT and refresh token from cache.	Set Cache-Control: no-store, no-cache on all token endpoint responses; verify Nginx/proxy configuration does not cache /token responses; add Pragma: no-cache and Expires: 0 as additional defence; test caching behaviour in integration tests.
140	Token signing key rotation causing mass token invalidation	Denial of service	Medium	Open		An uncoordinated signing key rotation in Keycloak immediately invalidates all currently issued tokens. All active users across ArcadeHaven are simultaneously logged out. If ArcadeHaven's Spring Security has a cached JWKS with the old key, it rejects all new tokens for the duration of the cache TTL as well.	Keycloak supports graceful key rotation with an overlap period — configure a key retirement delay (e.g. 1 hour) so old tokens remain valid during rollover; Spring Security JWKS cache TTL should be short enough to pick up the new key quickly (e.g. 15 min); test key rotation procedure in staging.

Token Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
137	Token payload tampered before signing — role claim injection	Tampering	Critical	Open		The Token Request flow carries the user's identity and role claims from Credential Validation to Issue Token. If this internal flow is interceptable (e.g. over an unencrypted message bus or shared memory), an attacker could inject an ADMIN role claim into the request payload before the token is signed, resulting in a legitimately signed token with fraudulent privileges.	For a standard Keycloak deployment this flow is in-process and not network-exposed — verify this is the case; if custom Keycloak SPIs or external mappers are used, ensure they communicate over secured channels; audit all custom Keycloak token mappers in the codebase for claim injection vulnerabilities.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
138	Issued token intercepted before delivery to client (token theft)	Tampering	Critical	Open		The Token Response carries the signed JWT back to the client. If delivered over HTTP (no TLS), an on-path attacker captures the access token and refresh token. They can now authenticate as the victim indefinitely (until the refresh token expires or is revoked).	Enforce HTTPS for all token delivery endpoints; HSTS header; refresh token delivered only in HttpOnly + Secure + SameSite=Strict cookie; access token in response body (short TTL ≤15 min); implement refresh token rotation so any interception is detectable via reuse detection
139	Token response cached by intermediate proxy — token leakage	Information disclosure	High	Open		A misconfigured reverse proxy or CDN caches the token endpoint response (missing Cache-Control: no-store). A subsequent request from a different user receives another user's JWT and refresh token from cache.	Set Cache-Control: no-store, no-cache on all token endpoint responses; verify Nginx/proxy configuration does not cache /token responses; add Pragma: no-cache and Expires: 0 as additional defence; test caching behaviour in integration tests.
140	Token signing key rotation causing mass token invalidation	Denial of service	Medium	Open		An uncoordinated signing key rotation in Keycloak immediately invalidates all currently issued tokens. All active users across ArcadeHaven are simultaneously logged out. If ArcadeHaven's Spring Security has a cached JWKS with the old key, it rejects all new tokens for the duration of the cache TTL as well.	Keycloak supports graceful key rotation with an overlap period — configure a key retirement delay (e.g. 1 hour) so old tokens remain valid during rollover; Spring Security JWKS cache TTL should be short enough to pick up the new key quickly (e.g. 15 min); test key rotation procedure in staging.

Token Validation Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
142	Token validation response tampered — invalid token accepted	Tampering	Critical	Open		The Token Validation Response carries a boolean valid: true/false decision back to User Management. If this internal flow is interceptable (e.g. a custom introspection implementation over an unprotected internal HTTP call), an attacker on the same network could flip valid: false to valid: true, causing an expired or revoked token to be accepted as legitimate, granting access to a compromised or logged-out session.	If using Keycloak's built-in local JWT validation (preferred), this flow is in-process and not network-exposed — use this approach; if using token introspection over HTTP, enforce mutual TLS on the introspection endpoint; never trust an external validation response without also independently verifying the JWT signature locally.
143	Token contents disclosed in validation error response	Information disclosure	High	Open		The Token Validation Response returns an error that includes the decoded token payload (e.g. "Token for user admin@arcadehaven.com expired at 14:32:00") to aid debugging. This leaks the victim's identity, email, and expiry details to anyone who can observe the validation response, including other application threads or log sinks.	Validation responses return only a result code (valid/invalid/expired) and never include decoded token contents; detailed failure reasons logged internally only with the token's jti (JWT ID), never the full payload; structured logging with explicit field allowlist.
144	Token validation flow flooded — all API requests blocked	Denial of service	High	Open		Every API request to ArcadeHaven triggers a Token Validation Request. An attacker submits thousands of requests per second with malformed JWTs. Even if each request is rejected quickly, the volume of validation flow requests saturates the processing capacity of the Token Validation process, causing all legitimate API requests to queue and time out.	Implement pre-validation at the network edge (e.g. Nginx rate-limiting) before the JWT reaches the validation flow; cache valid token results for short periods (e.g. 30 seconds per jti) to reduce repeated validations of the same token; circuit breaker around the validation process.
145	Token replay — revoked token re-submitted for validation	Tampering	High	Open		A user logs out; their token is supposed to be revoked. An attacker who captured the token before logout re-submits it through the Token Validation Request flow. If the validation process relies only on the JWT signature and expiry (stateless validation) and does not check a revocation list, the revoked token is accepted as valid.	Maintain a token revocation list (or use Keycloak's logout event propagation to Spring Security); for high-sensitivity operations, use token introspection rather than stateless validation; keep access token TTL very short (≤5 min) to limit the window for replay; implement Keycloak back-channel logout notifications.
146	Token submitted for validation logged in full	Information disclosure	Medium	Open		The Token Validation Request is logged at DEBUG level including the raw JWT string. Any developer or system with access to the log store can extract the token and use it to impersonate the user for the remaining token lifetime.	Never log raw JWT strings; log only the jti claim (unique token ID) for correlation purposes; apply log sanitisation rules that mask Authorization header values and bearer tokens in all log outputs; rotate logs with restricted access.
147	Slow introspection response causing ArcadeHaven API timeout cascade	Denial of service	Medium	Open		If token validation uses the Keycloak introspection endpoint over HTTP, a Keycloak slowdown causes the Token Validation Request flow to hang. Without a timeout, every ArcadeHaven API thread blocks waiting for a validation response, exhausting the thread pool and making the entire API unavailable — a Keycloak performance issue cascades into a full ArcadeHaven outage.	Set explicit timeouts on all calls in the validation flow (e.g. 500ms); prefer local JWT signature validation (no network call) over introspection for standard requests; use introspection only for logout/revocation checks; circuit breaker (Resilience4j) wrapping introspection calls with fallback to local validation.

Token Validation Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
142	Token validation response tampered — invalid token accepted	Tampering	Critical	Open		The Token Validation Response carries a boolean valid: true/false decision back to User Management. If this internal flow is interceptable (e.g. a custom introspection implementation over an unprotected internal HTTP call), an attacker on the same network could flip valid: false to valid: true, causing an expired or revoked token to be accepted as legitimate, granting access to a compromised or logged-out session.	If using Keycloak's built-in local JWT validation (preferred), this flow is in-process and not network-exposed — use this approach; if using token introspection over HTTP, enforce mutual TLS on the introspection endpoint; never trust an external validation response without also independently verifying the JWT signature locally.
143	Token contents disclosed in validation error response	Information disclosure	High	Open		The Token Validation Response returns an error that includes the decoded token payload (e.g. "Token for user admin@arcadehaven.com expired at 14:32:00") to aid debugging. This leaks the victim's identity, email, and expiry details to anyone who can observe the validation response, including other application threads or log sinks.	Validation responses return only a result code (valid/invalid/expired) and never include decoded token contents; detailed failure reasons logged internally only with the token's jti (JWT ID), never the full payload; structured logging with explicit field allowlist.
144	Token validation flow flooded — all API requests blocked	Denial of service	High	Open		Every API request to ArcadeHaven triggers a Token Validation Request. An attacker submits thousands of requests per second with malformed JWTs. Even if each request is rejected quickly, the volume of validation flow requests saturates the processing capacity of the Token Validation process, causing all legitimate API requests to queue and time out.	Implement pre-validation at the network edge (e.g. Nginx rate-limiting) before the JWT reaches the validation flow; cache valid token results for short periods (e.g. 30 seconds per jti) to reduce repeated validations of the same token; circuit breaker around the validation process.
145	Token replay — revoked token re-submitted for validation	Tampering	High	Open		A user logs out; their token is supposed to be revoked. An attacker who captured the token before logout re-submits it through the Token Validation Request flow. If the validation process relies only on the JWT signature and expiry (stateless validation) and does not check a revocation list, the revoked token is accepted as valid.	Maintain a token revocation list (or use Keycloak's logout event propagation to Spring Security); for high-sensitivity operations, use token introspection rather than stateless validation; keep access token TTL very short (≤5 min) to limit the window for replay; implement Keycloak back-channel logout notifications.
146	Token submitted for validation logged in full	Information disclosure	Medium	Open		The Token Validation Request is logged at DEBUG level including the raw JWT string. Any developer or system with access to the log store can extract the token and use it to impersonate the user for the remaining token lifetime.	Never log raw JWT strings; log only the jti claim (unique token ID) for correlation purposes; apply log sanitisation rules that mask Authorization header values and bearer tokens in all log outputs; rotate logs with restricted access.
147	Slow introspection response causing ArcadeHaven API timeout cascade	Denial of service	Medium	Open		If token validation uses the Keycloak introspection endpoint over HTTP, a Keycloak slowdown causes the Token Validation Request flow to hang. Without a timeout, every ArcadeHaven API thread blocks waiting for a validation response, exhausting the thread pool and making the entire API unavailable — a Keycloak performance issue cascades into a full ArcadeHaven outage.	Set explicit timeouts on all calls in the validation flow (e.g. 500ms); prefer local JWT signature validation (no network call) over introspection for standard requests; use introspection only for logout/revocation checks; circuit breaker (Resilience4j) wrapping introspection calls with fallback to local validation.

Role Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
148	Role Response tampered — privilege escalation via role injection	Tampering	Critical	Open		The Role Response carries the resolved role set (e.g. [BUYER]) back to Token Validation. If this flow is interceptable, an attacker modifies the response to inject [BUYER, ADMIN] before it is consumed by the access control decision logic. The result is a legitimate user transparently escalated to ADMIN without any Keycloak change. This is the single highest-impact threat in the entire Auth API data flow layer.	For a standard Keycloak deployment, role resolution is in-process — never externalise it over a network call; if a custom role resolver is used (e.g. a separate microservice), enforce mutual TLS and message signing on that service; in Spring Security, always extract roles directly from the cryptographically verified JWT claims, not from a separate service call whose response could be tampered with.
149	Role Response discloses full role hierarchy to unintended consumers	Information disclosure	High	Open		The Role Response includes not just the user's effective roles but also the full Keycloak composite role structure and internal role IDs. If this response is logged or returned to the ArcadeHaven application in full, it exposes Keycloak's internal role configuration to developers and log monitoring systems — providing an attacker who gains log access with a complete map of the privilege hierarchy to target.	Role Response should contain only the flat list of effective role names needed by the application (ADMIN, PUBLISHER, BUYER) — no internal Keycloak role IDs, composite structures, or metadata; never log the full role response; Spring Security's authority mapping should extract only the required claim paths.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
150	Complex composite role resolution causing authorisation latency spikes	Denial of service	Medium	Open		The Role Request triggers resolution of a deeply nested Keycloak composite role structure (e.g. ADMIN inherits from MANAGER which inherits from PUBLISHER which inherits from BUYER). Under load, each Role Request requires multiple recursive DB lookups to flatten the role hierarchy, causing authorisation latency spikes that degrade the entire API response time.	Keep the ArcadeHaven role hierarchy flat — 3 standalone roles (ADMIN, PUBLISHER, BUYER) with no composite inheritance; role resolution should be a single O(1) lookup from the JWT claim; test role resolution latency under load; cache resolved role sets per jti with a short TTL if composite roles cannot be avoided.
151	Role Request carries stale identity after token reuse	Tampering	High	Open		A Role Request is made using an old token jti whose associated user has since had their role downgraded from PUBLISHER to BUYER in Keycloak. The Claim Role process resolves roles from the token payload (which still says PUBLISHER) rather than re-querying Keycloak's current role assignments, returning the outdated PUBLISHER role in the Role Response.	Claim Role process must resolve roles from the live JWT claims (which encode the role at issuance time) — to get real-time role changes, keep token TTL very short (≤5 min) so re-authentication forces a fresh token with updated claims; for immediate revocation, use Keycloak back-channel logout or a role-change event that invalidates existing tokens.
152	Role denial reason leaks role names in 403 response	Information disclosure	Medium	Open		When Claim Role determines a user lacks a required role, the Role Response includes the required role name (e.g. "required: ADMIN, present: BUYER"). This propagates through Token Validation back to the API response as a 403 body containing "requires role ADMIN", confirming the exact role name an attacker needs to target for privilege escalation.	403 responses return only "Forbidden" with no role detail; the required role and present roles are logged internally (with the request jti for correlation) but never sent to the client; test that all 403 responses in integration tests contain no role information.
153	Role Request flow disrupted by Keycloak realm misconfiguration	Denial of service	Low	Open		A Keycloak admin accidentally deletes the ArcadeHaven realm's role definitions. All Role Requests return empty role sets. The Claim Role process returns no roles for every user, causing all authenticated users to be treated as having no permissions — effectively a complete authorisation outage.	Keycloak realm configuration (roles, clients, mappers) version-controlled as Infrastructure-as-Code (e.g. Keycloak realm export in the Git repository); automated deployment pipeline re-applies configuration; smoke test after any Keycloak configuration change that verifies role claims are present in issued tokens.

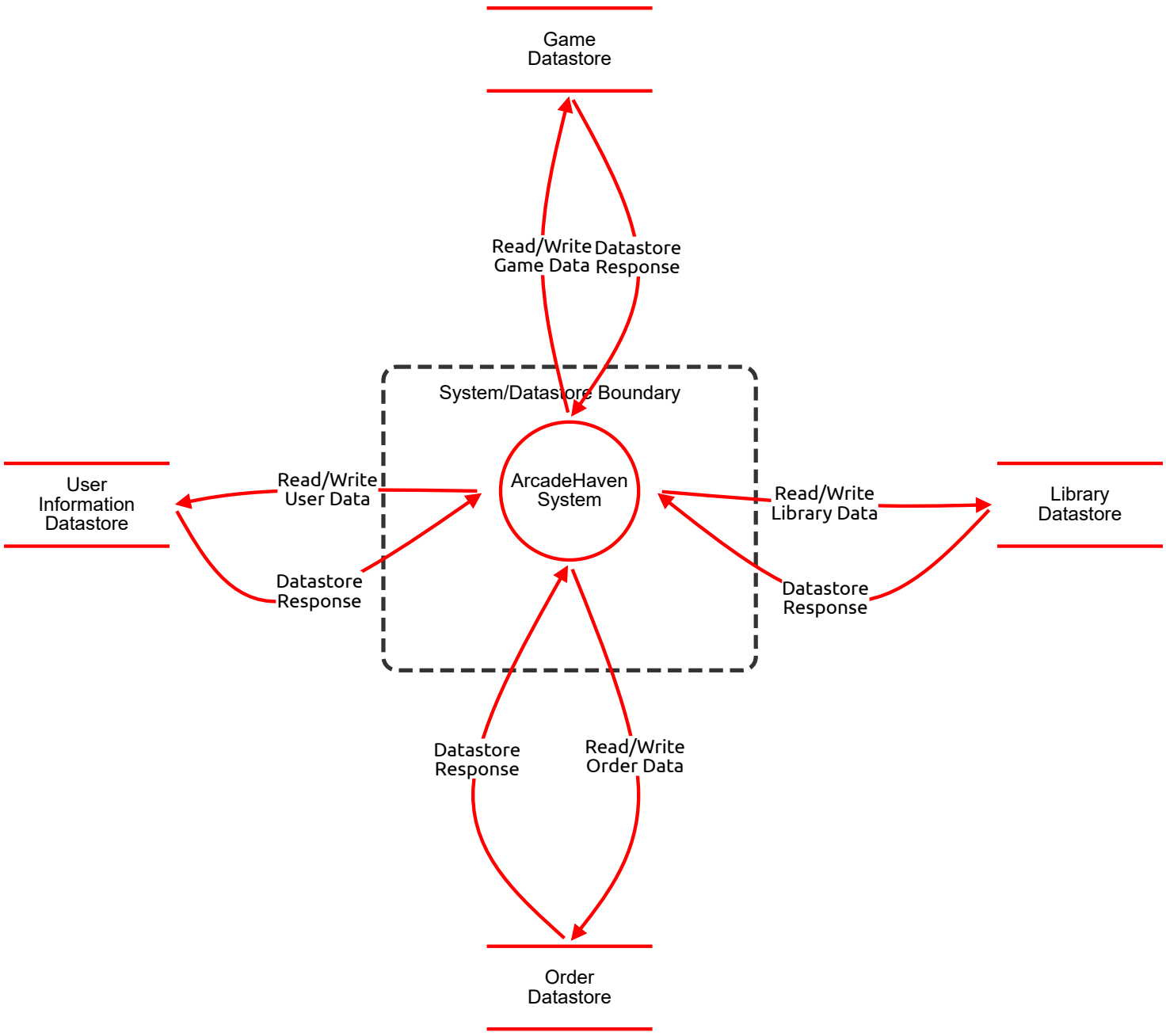
Role Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
148	Role Response tampered — privilege escalation via role injection	Tampering	Critical	Open		The Role Response carries the resolved role set (e.g. [BUYER]) back to Token Validation. If this flow is interceptable, an attacker modifies the response to inject [BUYER, ADMIN] before it is consumed by the access control decision logic. The result is a legitimate user transparently escalated to ADMIN without any Keycloak change. This is the single highest-impact threat in the entire Auth API data flow layer.	For a standard Keycloak deployment, role resolution is in-process — never externalise it over a network call; if a custom role resolver is used (e.g. a separate microservice), enforce mutual TLS and message signing on that service; in Spring Security, always extract roles directly from the cryptographically verified JWT claims, not from a separate service call whose response could be tampered with.
149	Role Response discloses full role hierarchy to unintended consumers	Information disclosure	High	Open		The Role Response includes not just the user's effective roles but also the full Keycloak composite role structure and internal role IDs. If this response is logged or returned to the ArcadeHaven application in full, it exposes Keycloak's internal role configuration to developers and log monitoring systems — providing an attacker who gains log access with a complete map of the privilege hierarchy to target.	Role Response should contain only the flat list of effective role names needed by the application (ADMIN, PUBLISHER, BUYER) — no internal Keycloak role IDs, composite structures, or metadata; never log the full role response; Spring Security's authority mapping should extract only the required claim paths.
150	Complex composite role resolution causing authorisation latency spikes	Denial of service	Medium	Open		The Role Request triggers resolution of a deeply nested Keycloak composite role structure (e.g. ADMIN inherits from MANAGER which inherits from PUBLISHER which inherits from BUYER). Under load, each Role Request requires multiple recursive DB lookups to flatten the role hierarchy, causing authorisation latency spikes that degrade the entire API response time.	Keep the ArcadeHaven role hierarchy flat — 3 standalone roles (ADMIN, PUBLISHER, BUYER) with no composite inheritance; role resolution should be a single O(1) lookup from the JWT claim; test role resolution latency under load; cache resolved role sets per jti with a short TTL if composite roles cannot be avoided.
151	Role Request carries stale identity after token reuse	Tampering	High	Open		A Role Request is made using an old token jti whose associated user has since had their role downgraded from PUBLISHER to BUYER in Keycloak. The Claim Role process resolves roles from the token payload (which still says PUBLISHER) rather than re-querying Keycloak's current role assignments, returning the outdated PUBLISHER role in the Role Response.	Claim Role process must resolve roles from the live JWT claims (which encode the role at issuance time) — to get real-time role changes, keep token TTL very short (≤5 min) so re-authentication forces a fresh token with updated claims; for immediate revocation, use Keycloak back-channel logout or a role-change event that invalidates existing tokens.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
152	Role denial reason leaks role names in 403 response	Information disclosure	Medium	Open		When Claim Role determines a user lacks a required role, the Role Response includes the required role name (e.g. "required: ADMIN, present: BUYER"). This propagates through Token Validation back to the API response as a 403 body containing "requires role ADMIN", confirming the exact role name an attacker needs to target for privilege escalation.	403 responses return only "Forbidden" with no role detail; the required role and present roles are logged internally (with the request jti for correlation) but never sent to the client; test that all 403 responses in integration tests contain no role information.
153	Role Request flow disrupted by Keycloak realm misconfiguration	Denial of service	Low	Open		A Keycloak admin accidentally deletes the ArcadeHaven realm's role definitions. All Role Requests return empty role sets. The Claim Role process returns no roles for every user, causing all authenticated users to be treated as having no permissions — effectively a complete authorisation outage.	Keycloak realm configuration (roles, clients, mappers) version-controlled as Infrastructure-as-Code (e.g. Keycloak realm export in the Git repository); automated deployment pipeline re-applies configuration; smoke test after any Keycloak configuration change that verifies role claims are present in issued tokens.

DFD - Level 1 - Database

Database interactions DFD



DFD - Level 1 - Database

ArcadeHaven System (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
8	Request body manipulation / mass assignment	Tampering	Medium	Open		Buyer sends a POST /orders with a modified price field, paying less for a game if the API trusts client-submitted prices instead of looking up the canonical price from the DB.	Never trust client-submitted prices; resolve price server-side from the Games datastore; use strict DTO validation with @Valid; disable mass assignment via explicit @JsonProperty mapping.
9	JWT not validated on every route	Spoofing	Critical	Open		Attacker crafts a request with an unsigned or expired token to a Spring endpoint that lacks @PreAuthorize; gains access as any user.	Global JWT validation filter in Spring Security; no endpoint bypasses; token expiry enforced; alg:none attack prevention (reject tokens with no algorithm).
10	Insufficient request logging	Repudiation	High	Open		An admin deletes a user account. No structured log exists with the admin's identity, timestamp, and target. They later deny the action.	Structured access log for all mutating operations (POST/PUT/PATCH/DELETE) including authenticated user ID, resource path, body hash, and timestamp; send logs to a tamper-evident SIEM.
12	Verbose error messages expose internals	Information disclosure	Critical	Open		A 500 error leaks a stack trace containing table names, ORM query structure, or DB credentials to the caller.	Global exception handler returning generic error responses; suppress stack traces in production; use structured error codes; never include DB exception messages in HTTP responses.
13	No rate limiting on public endpoints	Denial of service	Medium	Open		Attacker floods POST /orders or GET /games with thousands of requests/second, exhausting the Spring Boot thread pool and making the API unavailable to legitimate users.	API gateway or Spring rate-limiting filter (e.g. Bucket4j); limit per IP and per authenticated user; return 429 with Retry-After; circuit breaker pattern for downstream DB calls.
14	Missing role checks on admin routes	Elevation of privilege	Critical	Open		A Buyer calls DELETE /admin/games/{id} and it succeeds because the endpoint validates only authentication (valid JWT) but not the required ADMIN role in the Keycloak token claims.	Role-based access control enforced at the method level via @PreAuthorize("hasRole('ADMIN')"); integration tests for every protected endpoint verifying that Buyer/Publisher tokens are rejected; automated security scanning in CI pipeline with SonarQube.

Game Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
154	Direct price/status modification in DB	Tampering	High	Open		A malicious DBA or compromised DB account sets price = 0 for all games or sets status = ACTIVE for all PENDING games, bypassing the admin approval workflow.	Application DB user has INSERT/UPDATE/SELECT only on games table; no direct UPDATE on price/status columns (use stored procedures with audit); Flyway-managed schema prevents ad-hoc DDL.
155	No audit trail for game record changes	Repudiation	Medium	Open		A game's price is changed in the DB; no before/after record exists, making it impossible to prove what the price was at the time of a disputed purchase.	Trigger-based or application-level audit table (game_audit) capturing old/new values on UPDATE; append-only; include DB user and timestamp.
156	Internal game metadata exposed (e.g. unpublished games)	Information disclosure	High	Open		Internal game metadata exposed (e.g. unpublished games)	A query with missing WHERE status = 'ACTIVE' returns PENDING and DRAFT games including unreleased titles and internal notes to an unauthenticated caller.
157	Full-table scan on game catalogue	Denial of service	Medium	Open		A search query with no index on title or category columns causes a sequential scan across all games, holding locks and blocking concurrent writes from publishers.	Indices on games(title), games(category_id), games(status), games(publisher_id); EXPLAIN ANALYZE on all catalogue queries; pg_stat_statements monitoring for slow queries.

Library Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
158	Inserting library entries without a corresponding order	Tampering	High	Open		Direct INSERT into library_entries for a user_id / game_id pair without a corresponding completed order record; user gets a game for free.	Foreign key constraint: library_entries.order_id REFERENCES orders(id); application enforces entry creation only via order completion event; DB user cannot INSERT without a valid order FK.
159	No record of activation key usage	Repudiation	High	Open		An activation key is marked as used (activated = true) in the DB; no log shows when, from which IP, or by which session it was activated. Disputed activations cannot be investigated.	Append-only activation_events table recording key ID, user ID, timestamp, and IP address on each activation attempt; never UPDATE without inserting an event row.
160	Activation keys stored in plaintext	Information disclosure	Critical	Open		DB dump or a SQL injection attack exposes all activation keys in plaintext; attacker activates games without purchasing them.	Encrypt activation keys at application layer (AES-256) before storing; only decrypt when delivering to the authenticated owner; key management via environment secrets.
161	Library entry table growth degrading queries	Denial of service	Medium	Open		Over time, library_entries grows to millions of rows with no archival strategy; per-user library queries slow significantly.	Index on library_entries(user_id, game_id); partition table by user_id range if needed at scale; consider archiving entries for deactivated accounts.

Order Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
162	Order record retroactive modification	Tampering	Critical	Open		A DBA modifies order.total_price to 0.01 post-purchase; the stored invoice PDF and the DB record now disagree, creating a fraudulent audit trail.	Orders table is insert-only after completion; no UPDATE permitted on completed order records; use CHECK constraints or triggers to prevent status regression; invoice PDF hash stored in DB and verified on retrieval.
163	No payment/transaction audit trail	Repudiation	High	Open		An order's payment status changes from PENDING to COMPLETED; no event record exists with timestamp, amount, and payment reference, making financial reconciliation impossible.	Immutable order_events / payment_events table; every state transition appends a new row; never UPDATE status directly; financial data retained per compliance requirements (typically 7 years).
164	Order history cross-user exposure via missing WHERE clause	Information disclosure	High	Open		ORM query SELECT * FROM orders JOIN order_items ON ... missing WHERE buyer_id = :userId returns all users' order history to the first caller.	All order queries parameterised with buyer_id from JWT sub; JPA repository method signatures enforce the filter; integration tests verifying cross-user isolation.
165	Large order history queries degrading performance	Denial of service	High	Open		GET /orders with no date range filter loads all historical orders for a user or (worse) all users into memory.	Index on orders(buyer_id, created_at); mandatory date range or pagination on order history queries; max result window enforced.

User Information Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
166	Direct role/status manipulation in users table	Tampering	Critical	Open		A compromised DB account executes UPDATE users SET role = 'ADMIN' WHERE id = 42, granting admin access outside of Keycloak, if a local roles cache table is used.	If a local user table exists, the role column should be read-only to the application DB user; roles are always the authoritative source from the Keycloak JWT, never from a local DB cache.
167	No history of PII changes	Repudiation	High	Open		A user changes their email address; the users table UPDATE overwrites the old value with no history. A GDPR data subject request or fraud investigation cannot show what email was on file at a given date.	Separate user_profile_history table capturing old/new values on UPDATE with timestamp; or use event sourcing for the User aggregate; required for GDPR compliance (data accuracy and audit).
168	PII exposed via unencrypted DB dump	Information disclosure	High	Open		A DB backup exported in plaintext to an S3 bucket with misconfigured public access exposes all user emails, names, and addresses.	Encrypt DB backups at rest; restrict backup bucket access to infrastructure-only IAM roles; encrypt PII columns (email, address) at application layer; audit S3 bucket policies regularly.
169	User table lock contention during bulk operations	Denial of service	Medium	Open		A bulk user import or admin report query holds a table-level lock on users, blocking login queries that need to read user status.	Use SELECT ... FOR SHARE (row-level) rather than table locks; run bulk operations in off-peak windows; index on users(email) and users(keycloak_sub) for fast lookup.

Read/Write Game Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
170	SQL injection via game search / filter parameters	Tampering	Critical	Open		A publisher submits a game title containing a SQL fragment: Action' OR 1=1; DROP TABLE games;--. If the Read/Write Game Data flow uses a native query with string concatenation, the injected SQL executes, potentially destroying the entire games table or exfiltrating all records including unpublished, internal-notes, and pricing data.	Exclusively use JPA parameterised queries or Spring Data repository methods for all game data operations; ban native query string concatenation via SonarQube rule; enforce parameterisation in code review checklist; automated SQL injection tests in CI pipeline.
171	Read flow returns PENDING / DRAFT games to public callers	Information disclosure	High	Open		The Read Game Data flow executes a query without a WHERE status = 'ACTIVE' filter. The datastore response includes PENDING and DRAFT game records — containing unreleased titles, internal notes, cost prices, and publisher strategy data — which are then returned to unauthenticated or Buyer-role callers.	All public-facing read queries hardcode the status = 'ACTIVE' filter at the repository layer; separate repository methods for admin (all statuses) and public (ACTIVE only); integration test verifying PENDING/DRAFT games are never present in Buyer-role responses.
172	Full-table scan on game catalogue blocking concurrent writes	Denial of service	High	Open		A read query with no index on the title or category_id column triggers a sequential scan across the entire games table. The scan acquires a shared lock that blocks concurrent INSERT/UPDATE operations from publishers, causing write timeouts and failed game submissions during peak traffic.	Indices on games(title), games(category_id), games(status), games(publisher_id); EXPLAIN ANALYZE on all catalogue queries during development; pg_stat_statements monitoring for slow queries in production; read replica for heavy catalogue scans.
173	Game status field written directly to ACTIVE without approval	Tampering	High	Open		The write flow for game data accepts a status field from the application layer without verifying that the caller holds the ADMIN role. A Publisher-triggered update writes status = ACTIVE directly to the datastore, bypassing the PENDING → admin approval → ACTIVE workflow entirely.	The write flow must enforce role-based field filtering: the status field is excluded from all Publisher-facing write DTOs; only AdminGameUpdateDTO includes status; the query is constructed from the DTO, making it structurally impossible to write status from a Publisher request.
174	Read response includes internal-only fields (cost price, admin flags)	Information disclosure	Medium	Open		The datastore response returns a full entity object including cost_price, internal_notes, and admin_review_flag columns. These are serialised by the ORM into the application response DTO because SELECT * is used instead of a column projection, leaking internal business data to the API caller.	Use Spring Data projection interfaces or explicit column lists in queries — never SELECT *; separate public and admin response DTOs; @JsonIgnore on internal-only entity fields; integration test asserting internal fields are absent from public API responses.
175	Unbounded catalogue read exhausting JVM heap	Denial of service	Medium	Open		A read request with no pagination causes the datastore to return all game records in a single response. The ORM loads every record into memory simultaneously. As the catalogue grows to thousands of titles, this causes a JVM OutOfMemoryError that crashes the application.	Mandatory Pageable parameter on all game list repository methods; server-side maximum page size enforced (e.g. max 100 results); query timeout configured in HikariCP; never use findAll() without pagination on the games table.

Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
170	SQL injection via game search / filter parameters	Tampering	Critical	Open		A publisher submits a game title containing a SQL fragment: Action' OR 1=1; DROP TABLE games;--. If the Read/Write Game Data flow uses a native query with string concatenation, the injected SQL executes, potentially destroying the entire games table or exfiltrating all records including unpublished, internal-notes, and pricing data.	Exclusively use JPA parameterised queries or Spring Data repository methods for all game data operations; ban native query string concatenation via SonarQube rule; enforce parameterisation in code review checklist; automated SQL injection tests in CI pipeline.
171	Read flow returns PENDING / DRAFT games to public callers	Information disclosure	High	Open		The Read Game Data flow executes a query without a WHERE status = 'ACTIVE' filter. The datastore response includes PENDING and DRAFT game records — containing unreleased titles, internal notes, cost prices, and publisher strategy data — which are then returned to unauthenticated or Buyer-role callers.	All public-facing read queries hardcode the status = 'ACTIVE' filter at the repository layer; separate repository methods for admin (all statuses) and public (ACTIVE only); integration test verifying PENDING/DRAFT games are never present in Buyer-role responses.
172	Full-table scan on game catalogue blocking concurrent writes	Denial of service	High	Open		A read query with no index on the title or category_id column triggers a sequential scan across the entire games table. The scan acquires a shared lock that blocks concurrent INSERT/UPDATE operations from publishers, causing write timeouts and failed game submissions during peak traffic.	Indices on games(title), games(category_id), games(status), games(publisher_id); EXPLAIN ANALYZE on all catalogue queries during development; pg_stat_statements monitoring for slow queries in production; read replica for heavy catalogue scans.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
173	Game status field written directly to ACTIVE without approval	Tampering	High	Open		The write flow for game data accepts a status field from the application layer without verifying that the caller holds the ADMIN role. A Publisher-triggered update writes status = ACTIVE directly to the datastore, bypassing the PENDING → admin approval → ACTIVE workflow entirely.	The write flow must enforce role-based field filtering: the status field is excluded from all Publisher-facing write DTOs; only AdminGameUpdateDTO includes status; the query is constructed from the DTO, making it structurally impossible to write status from a Publisher request.
174	Read response includes internal-only fields (cost price, admin flags)	Information disclosure	Medium	Open		The datastore response returns a full entity object including cost_price, internal_notes, and admin_review_flag columns. These are serialised by the ORM into the application response DTO because SELECT * is used instead of a column projection, leaking internal business data to the API caller.	Use Spring Data projection interfaces or explicit column lists in queries — never SELECT *; separate public and admin response DTOs; @JsonIgnore on internal-only entity fields; integration test asserting internal fields are absent from public API responses.
175	Unbounded catalogue read exhausting JVM heap	Denial of service	Medium	Open		A read request with no pagination causes the datastore to return all game records in a single response. The ORM loads every record into memory simultaneously. As the catalogue grows to thousands of titles, this causes a JVM OutOfMemoryError that crashes the application.	Mandatory Pageable parameter on all game list repository methods; server-side maximum page size enforced (e.g. max 100 results); query timeout configured in HikariCP; never use findAll() without pagination on the games table.

Read/Write Library Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
176	Library entry written without a corresponding completed order	Tampering	Critical	Open		The write flow to the Library Datastore does not enforce a foreign key relationship to a completed order. An attacker (or a bug) issues a direct INSERT into library_entries for any user_id / game_id pair. The user gains access to any game in the catalogue without payment — the entire monetisation model is bypassed.	Foreign key constraint: library_entries.order_id REFERENCES orders(id) NOT NULL; the DB enforces that no library entry can exist without a completed order; application-layer rule: library write flow is only ever triggered by an order-completion event, never by a direct API call; integration test attempting direct library write without an order must be rejected at DB level.
177	Activation keys returned in plaintext in datastore response	Information disclosure	Critical	Open		The read flow retrieves library entries including the activation_key column in plaintext. If this response is logged at DEBUG level, cached by an intermediate layer, or returned to an API caller without ownership verification, the activation key — which has real monetary value and can be used by anyone on a gaming platform — is exposed to unintended parties.	Activation keys encrypted at application layer (AES-256) before writing to the datastore; read flow decrypts only when delivering to the verified owner; keys never included in list responses — only in a dedicated single-record endpoint with ownership check and rate limiting; keys never logged in any format.
178	Library table growth causing per-user query degradation	Denial of service	Medium	Open		As the platform scales, library_entries grows to tens of millions of rows. A user with a large game collection triggers a read flow that, despite the user_id filter, still scans a large portion of the table due to a missing composite index on (user_id, game_id), causing slow query times that degrade the library browsing experience platform-wide.	Composite index on library_entries(user_id, game_id); covering index including activated and created_at for common sort/filter operations; EXPLAIN ANALYZE on library queries at realistic data volumes; consider table partitioning by user_id range at scale.
179	Activation key marked as used without actual activation	Tampering	High	Open		The write flow updates library_entries.activated = true for a key that has not been legitimately activated. If an attacker can trigger this write (e.g. via a race condition or a missing ownership check), the real owner's key is burned — they lose the ability to activate their purchased game even though they never used the key themselves.	Activation write flow requires re-authentication confirmation before marking a key as used; activation is a one-time, irreversible operation — add a DB CHECK constraint preventing reactivation (activated can only transition false → true, never back); append an activation_events record with timestamp and IP on every activation attempt.
180	Library read flow missing user_id scope — cross-user data exposure	Information disclosure	High	Open		The read flow for library data executes SELECT * FROM library_entries WHERE game_id = :gameId without scoping to the authenticated user's ID. A Buyer who knows any game_id can retrieve the library entries (and activation keys) of every user who owns that game.	All library read queries must include AND user_id = :authenticatedUserId sourced from the JWT sub; Spring Data repository method signatures enforce the user_id parameter — it cannot be omitted; automated test with two different user JWTs verifying each sees only their own entries.
181	Concurrent activation writes causing DB lock contention	Denial of service	Low	Open		causing DB lock contention Multiple simultaneous activation requests for the same library entry (e.g. a user double-clicking the activate button) create concurrent UPDATE statements on the same row, causing lock contention and potential duplicate activation event records.	Optimistic locking on library_entries using a @Version field; the second concurrent write fails with an OptimisticLockException and returns a 409 Conflict; idempotency check: if activated = true already, return success without a second write.

Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
176	Library entry written without a corresponding completed order	Tampering	Critical	Open		The write flow to the Library Datastore does not enforce a foreign key relationship to a completed order. An attacker (or a bug) issues a direct INSERT into library_entries for any user_id / game_id pair. The user gains access to any game in the catalogue without payment — the entire monetisation model is bypassed.	Foreign key constraint: library_entries.order_id REFERENCES orders(id) NOT NULL; the DB enforces that no library entry can exist without a completed order; application-layer rule: library write flow is only ever triggered by an order-completion event, never by a direct API call; integration test attempting direct library write without an order must be rejected at DB level.
177	Activation keys returned in plaintext in datastore response	Information disclosure	Critical	Open		The read flow retrieves library entries including the activation_key column in plaintext. If this response is logged at DEBUG level, cached by an intermediate layer, or returned to an API caller without ownership verification, the activation key — which has real monetary value and can be used by anyone on a gaming platform — is exposed to unintended parties.	Activation keys encrypted at application layer (AES-256) before writing to the datastore; read flow decrypts only when delivering to the verified owner; keys never included in list responses — only in a dedicated single-record endpoint with ownership check and rate limiting; keys never logged in any format.
178	Library table growth causing per-user query degradation	Denial of service	Medium	Open		As the platform scales, library_entries grows to tens of millions of rows. A user with a large game collection triggers a read flow that, despite the user_id filter, still scans a large portion of the table due to a missing composite index on (user_id, game_id), causing slow query times that degrade the library browsing experience platform-wide.	Composite index on library_entries(user_id, game_id); covering index including activated and created_at for common sort/filter operations; EXPLAIN ANALYZE on library queries at realistic data volumes; consider table partitioning by user_id range at scale.
179	Activation key marked as used without actual activation	Tampering	High	Open		The write flow updates library_entries.activated = true for a key that has not been legitimately activated. If an attacker can trigger this write (e.g. via a race condition or a missing ownership check), the real owner's key is burned — they lose the ability to activate their purchased game even though they never used the key themselves.	Activation write flow requires re-authentication confirmation before marking a key as used; activation is a one-time, irreversible operation — add a DB CHECK constraint preventing reactivation (activated can only transition false → true, never back); append an activation_events record with timestamp and IP on every activation attempt.
180	Library read flow missing user_id scope — cross-user data exposure	Information disclosure	High	Open		The read flow for library data executes SELECT * FROM library_entries WHERE game_id = :gameId without scoping to the authenticated user's ID. A Buyer who knows any game_id can retrieve the library entries (and activation keys) of every user who owns that game.	All library read queries must include AND user_id = :authenticatedUserId sourced from the JWT sub; Spring Data repository method signatures enforce the user_id parameter — it cannot be omitted; automated test with two different user JWTs verifying each sees only their own entries.
181	Concurrent activation writes causing DB lock contention	Denial of service	Low	Open		causing DB lock contention Multiple simultaneous activation requests for the same library entry (e.g. a user double-clicking the activate button) create concurrent UPDATE statements on the same row, causing lock contention and potential duplicate activation event records.	Optimistic locking on library_entries using a @Version field; the second concurrent write fails with an OptimisticLockException and returns a 409 Conflict; idempotency check: if activated = true already, return success without a second write.

Read/Write Order Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
182	Completed order record modified retroactively	Tampering	Critical	Open		A write flow updates an existing completed order record — changing total_price from €59.99 to €0.01 or setting status back to PENDING. The stored invoice PDF and the DB record now disagree. This constitutes financial fraud, violates accounting integrity, and makes the system's financial records untrustworthy for tax and compliance audits.	Orders table is effectively insert-only after completion: no UPDATE permitted on completed order records; DB-level trigger raises an exception on any UPDATE to orders WHERE status = 'COMPLETED'; CHECK constraint preventing status regression (COMPLETED cannot revert to PENDING); invoice PDF SHA-256 hash stored alongside the order record and verified on every retrieval.
183	Order datastore response exposes other users' financial records	Information disclosure	Critical	Open		The read flow for order data executes a query scoped only to order_id (e.g. SELECT * FROM orders WHERE id = :orderId) without including AND buyer_id = :jwtUserId. Any authenticated user who knows or can guess an order ID receives the complete financial record — including payment method reference, total paid, invoice, and activation key — of another user's purchase.	Every order read query mandatorily scopes to the authenticated user's buyer_id from the JWT sub; no order can be retrieved by ID alone without the ownership check; ADMIN-role queries use a separate repository method; automated IDOR test: user A's JWT must not retrieve user B's order ID.
184	Order write flow flooded — DB connection pool exhausted	Denial of service	High	Open		An attacker or bot creates hundreds of orders per second using a valid Buyer JWT. Each order creation triggers a DB write transaction (order + order_items + library_entry + activation_key generation). The volume exhausts HikariCP's connection pool, blocking all other DB operations including game catalogue reads and authentication lookups.	Rate-limit order creation per authenticated user (e.g. max 5 orders per minute); per-user circuit breaker on the order write flow; async activation key generation decoupled from the synchronous order write; alert on anomalous order creation rates; HikariCP pool sized with headroom and per-subsystem connection limits.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
185	Price written to order from client-supplied value instead of DB lookup	Tampering	Critical	Open		The write flow for order creation accepts a total_price field from the application layer that originated from the client request body (€0.01 for a €59.99 game). The flow writes this attacker-supplied price directly to the Order Datastore without re-validating it against the Games Datastore's canonical price at the time of purchase.	Order write flow always reads the authoritative price from the Game Datastore immediately before writing the order record — the price is never sourced from anywhere else; the order DTO accepted from the API layer contains no price field; price is resolved server-side and included only in the DB write, never round-tripped through the client.
186	Payment reference / financial data logged from datastore response	Information disclosure	High	Open		The application logs the full order object returned by the datastore response at INFO level for debugging. The log entry includes the payment gateway reference number, total amount paid, and buyer email — violating PCI-DSS and GDPR requirements and creating a high-value target for anyone with log access.	Never log full order objects; log only order_id and status for operational tracking; payment reference numbers and financial amounts are excluded from all log output via a custom Jackson serialiser with @JsonIgnore on sensitive fields used in logging contexts; structured logging with an explicit field allowlist.
187	Large order history read causing memory pressure	Denial of service	Medium	Open		A user with years of purchase history calls GET /orders with no date range filter. The read flow returns all historical orders with their order_items in a single response, loading thousands of records (and their JPA associations) into heap memory simultaneously.	Mandatory date range or pagination on all order history read queries; max page size enforced at repository level; index on orders(buyer_id, created_at DESC) for efficient paginated retrieval; lazy-load order_items association by default.

Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
182	Completed order record modified retroactively	Tampering	Critical	Open		A write flow updates an existing completed order record — changing total_price from €59.99 to €0.01 or setting status back to PENDING. The stored invoice PDF and the DB record now disagree. This constitutes financial fraud, violates accounting integrity, and makes the system's financial records untrustworthy for tax and compliance audits.	Orders table is effectively insert-only after completion: no UPDATE permitted on completed order records; DB-level trigger raises an exception on any UPDATE to orders WHERE status = 'COMPLETED'; CHECK constraint preventing status regression (COMPLETED cannot revert to PENDING); invoice PDF SHA-256 hash stored alongside the order record and verified on every retrieval.
183	Order datastore response exposes other users' financial records	Information disclosure	Critical	Open		The read flow for order data executes a query scoped only to order_id (e.g. SELECT * FROM orders WHERE id = :orderId) without including AND buyer_id = :jwtUserId. Any authenticated user who knows or can guess an order ID receives the complete financial record — including payment method reference, total paid, invoice, and activation key — of another user's purchase.	Every order read query mandatorily scopes to the authenticated user's buyer_id from the JWT sub; no order can be retrieved by ID alone without the ownership check; ADMIN-role queries use a separate repository method; automated IDOR test: user A's JWT must not retrieve user B's order ID.
184	Order write flow flooded — DB connection pool exhausted	Denial of service	High	Open		An attacker or bot creates hundreds of orders per second using a valid Buyer JWT. Each order creation triggers a DB write transaction (order + order_items + library_entry + activation_key generation). The volume exhausts HikariCP's connection pool, blocking all other DB operations including game catalogue reads and authentication lookups.	Rate-limit order creation per authenticated user (e.g. max 5 orders per minute); per-user circuit breaker on the order write flow; async activation key generation decoupled from the synchronous order write; alert on anomalous order creation rates; HikariCP pool sized with headroom and per-subsystem connection limits.
185	Price written to order from client-supplied value instead of DB lookup	Tampering	Critical	Open		The write flow for order creation accepts a total_price field from the application layer that originated from the client request body (€0.01 for a €59.99 game). The flow writes this attacker-supplied price directly to the Order Datastore without re-validating it against the Games Datastore's canonical price at the time of purchase.	Order write flow always reads the authoritative price from the Game Datastore immediately before writing the order record — the price is never sourced from anywhere else; the order DTO accepted from the API layer contains no price field; price is resolved server-side and included only in the DB write, never round-tripped through the client.
186	Payment reference / financial data logged from datastore response	Information disclosure	High	Open		The application logs the full order object returned by the datastore response at INFO level for debugging. The log entry includes the payment gateway reference number, total amount paid, and buyer email — violating PCI-DSS and GDPR requirements and creating a high-value target for anyone with log access.	Never log full order objects; log only order_id and status for operational tracking; payment reference numbers and financial amounts are excluded from all log output via a custom Jackson serialiser with @JsonIgnore on sensitive fields used in logging contexts; structured logging with an explicit field allowlist.
187	Large order history read causing memory pressure	Denial of service	Medium	Open		A user with years of purchase history calls GET /orders with no date range filter. The read flow returns all historical orders with their order_items in a single response, loading thousands of records (and their JPA associations) into heap memory simultaneously.	Mandatory date range or pagination on all order history read queries; max page size enforced at repository level; index on orders(buyer_id, created_at DESC) for efficient paginated retrieval; lazy-load order_items association by default.

Read/Write User Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
188	Email address written without ownership verification — account takeover enablement	Tampering	Critical	Open		The write flow for user data accepts an email update and writes it directly to the User Information Datastore without verifying that the new email belongs to the authenticated user. An attacker changes their email to an admin's address. The password reset flow now sends a reset link to the attacker's inbox, granting full account takeover of the admin account.	Email change write flow requires: (1) re-authentication with current password, (2) sending a verification link to the NEW email address that must be clicked before the write is committed, (3) notification to the OLD email address that a change was requested; only after step 2 completes does the write flow update the datastore; old email stored in history table.
189	User datastore response returns full PII to non-owner callers	Information disclosure	Critical	Open		The read flow returns the full User entity from the datastore including name, email, address, date of birth, Keycloak subject ID, and account_status to any authenticated caller who supplies a valid user_id — even if that user_id belongs to a different user. A Buyer enumerates user IDs sequentially and harvests the PII of every registered user, constituting a GDPR data breach.	Read flow must enforce: authenticated user can only read their own record (JWT sub = target user_id) OR caller holds ADMIN role; admin responses use a separate DTO that still excludes Keycloak internal fields; Buyer-facing responses use a minimal DTO containing only display name and email; automated IDOR test with two different user JWTs is mandatory in CI.
190	User table lock contention during bulk admin operations	Denial of service	High	Open		An admin report or bulk user export triggers a long-running read across the entire users table (SELECT * FROM users with no pagination). The query holds a shared lock that blocks concurrent write flows — specifically login-time user status updates and profile writes — causing login failures and profile update timeouts for all active users during the operation.	Bulk admin read operations run on a read replica, never the primary DB; LIMIT/OFFSET or cursor-based pagination on all admin user list queries; index on users(email) and users(keycloak_sub) for point lookups; schedule heavy report queries during off-peak hours; query timeout on all admin queries (e.g. 30 seconds maximum).
191	SQL injection via user profile search or filter — full user table exfiltration	Tampering	Critical	Open		An admin search for users by email uses a native query with string concatenation: SELECT * FROM users WHERE email LIKE '%' + :input + '%'. An injected payload UNION SELECT username, password_hash, keycloak_sub FROM users-- returns the entire user table including all Keycloak subject IDs and any locally cached sensitive fields to the admin UI.	All user data queries use JPA parameterised queries; LIKE patterns use CONCAT(:param, '%') with a bound parameter, never string concatenation; even admin-facing queries follow the same parameterisation rules — privilege does not exempt from SQL injection protection.
192	User datastore response logged — PII written to application logs (GDPR violation)	Information disclosure	Critical	Open		The application logs the User entity object returned by the datastore response at INFO level for audit purposes. The log entry contains name, email, address, and Keycloak subject ID for every user whose profile is accessed. Log aggregation tools (ELK, Splunk) now hold a full copy of all user PII — a separate, uncontrolled data store that violates GDPR's data minimisation and purpose limitation principles.	User entity must implement toString() returning only user_id and account_status — never PII fields; @JsonIgnore on all PII fields in the entity; structured logging records only user_id for operational tracking; PII fields are explicitly excluded from all log formatters; GDPR data mapping must include the logging pipeline as a data processing activity.
193	User enumeration via sequential ID read flood exhausting DB connections	Denial of service	Medium	Open		An attacker sends thousands of GET /users/{id} requests with sequential integer IDs. Even though each request is rejected (IDOR check fails), the read flow still executes a DB query per request to perform the lookup before applying the ownership check. The volume of queries exhausts DB connections and degrades response times for all other flows.	Apply the ownership check before the DB query where possible (e.g. compare JWT sub to URL parameter before hitting the DB — if they don't match, return 403 immediately without a DB round-trip); rate-limit user profile reads per authenticated user; IP-level rate limiting at the reverse proxy for unauthenticated enumeration attempts.

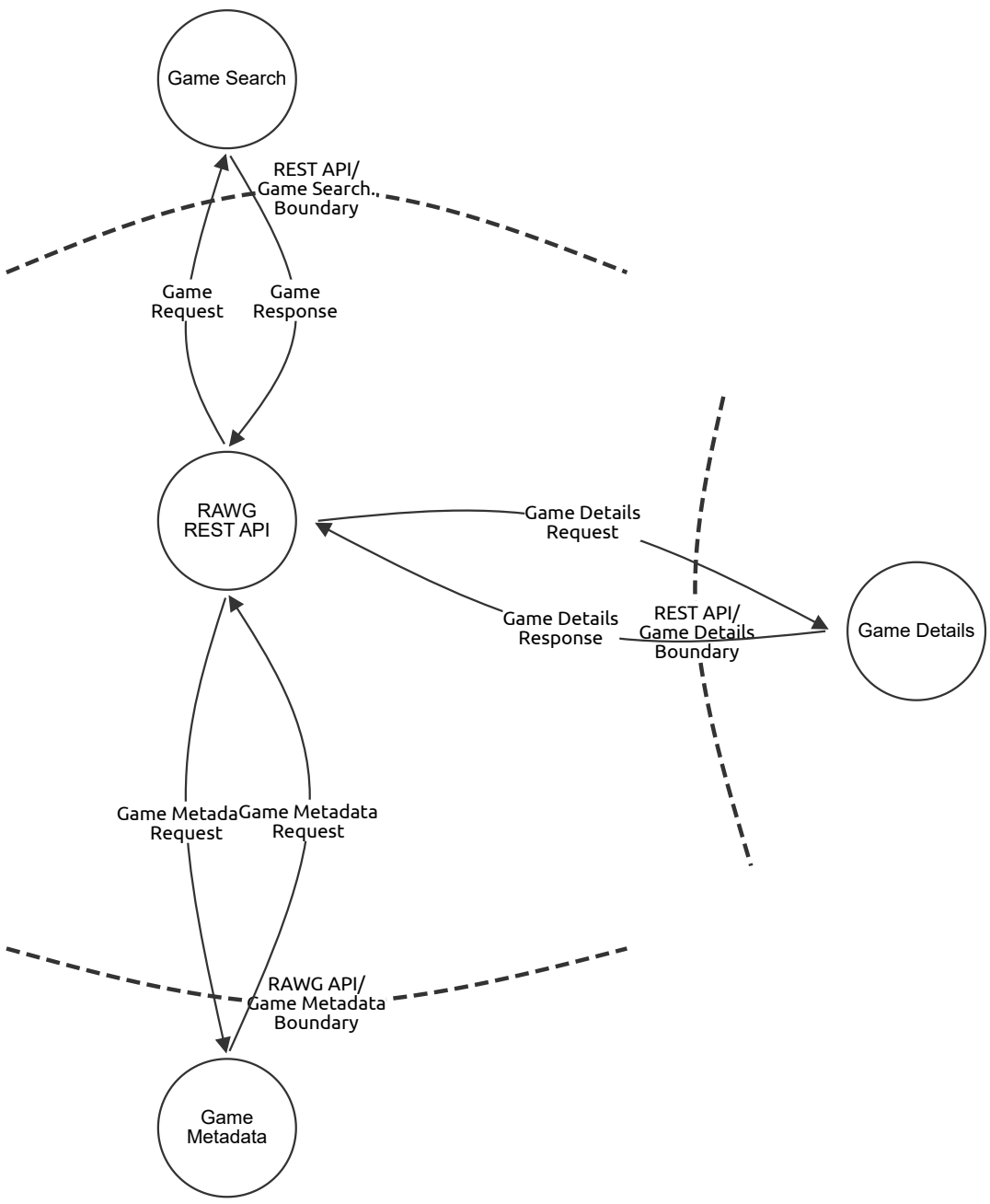
Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
188	Email address written without ownership verification — account takeover enablement	Tampering	Critical	Open		The write flow for user data accepts an email update and writes it directly to the User Information Datastore without verifying that the new email belongs to the authenticated user. An attacker changes their email to an admin's address. The password reset flow now sends a reset link to the attacker's inbox, granting full account takeover of the admin account.	Email change write flow requires: (1) re-authentication with current password, (2) sending a verification link to the NEW email address that must be clicked before the write is committed, (3) notification to the OLD email address that a change was requested; only after step 2 completes does the write flow update the datastore; old email stored in history table.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
189	User datastore response returns full PII to non-owner callers	Information disclosure	Critical	Open		The read flow returns the full User entity from the datastore including name, email, address, date of birth, Keycloak subject ID, and account_status to any authenticated caller who supplies a valid user_id — even if that user_id belongs to a different user. A Buyer enumerates user IDs sequentially and harvests the PII of every registered user, constituting a GDPR data breach.	Read flow must enforce: authenticated user can only read their own record (JWT sub = target user_id) OR caller holds ADMIN role; admin responses use a separate DTO that still excludes Keycloak internal fields; Buyer-facing responses use a minimal DTO containing only display name and email; automated IDOR test with two different user JWTs is mandatory in CI.
190	User table lock contention during bulk admin operations	Denial of service	High	Open		An admin report or bulk user export triggers a long-running read across the entire users table (SELECT * FROM users with no pagination). The query holds a shared lock that blocks concurrent write flows — specifically login-time user status updates and profile writes — causing login failures and profile update timeouts for all active users during the operation.	Bulk admin read operations run on a read replica, never the primary DB; LIMIT/OFFSET or cursor-based pagination on all admin user list queries; index on users(email) and users(keycloak_sub) for point lookups; schedule heavy report queries during off-peak hours; query timeout on all admin queries (e.g. 30 seconds maximum).
191	SQL injection via user profile search or filter — full user table exfiltration	Tampering	Critical	Open		An admin search for users by email uses a native query with string concatenation: SELECT * FROM users WHERE email LIKE '%' + :input + '%'. An injected payload UNION SELECT username, password_hash, keycloak_sub FROM users-- returns the entire user table including all Keycloak subject IDs and any locally cached sensitive fields to the admin UI.	All user data queries use JPA parameterised queries; LIKE patterns use CONCAT(:param, '%') with a bound parameter, never string concatenation; even admin-facing queries follow the same parameterisation rules — privilege does not exempt from SQL injection protection.
192	User datastore response logged — PII written to application logs (GDPR violation)	Information disclosure	Critical	Open		The application logs the User entity object returned by the datastore response at INFO level for audit purposes. The log entry contains name, email, address, and Keycloak subject ID for every user whose profile is accessed. Log aggregation tools (ELK, Splunk) now hold a full copy of all user PII — a separate, uncontrolled data store that violates GDPR's data minimisation and purpose limitation principles.	User entity must implement toString() returning only user_id and account_status — never PII fields; @JsonIgnore on all PII fields in the entity; structured logging records only user_id for operational tracking; PII fields are explicitly excluded from all log formatters; GDPR data mapping must include the logging pipeline as a data processing activity.
193	User enumeration via sequential ID read flood exhausting DB connections	Denial of service	Medium	Open		An attacker sends thousands of GET /users/{id} requests with sequential integer IDs. Even though each request is rejected (IDOR check fails), the read flow still executes a DB query per request to perform the lookup before applying the ownership check. The volume of queries exhausts DB connections and degrades response times for all other flows.	Apply the ownership check before the DB query where possible (e.g. compare JWT sub to URL parameter before hitting the DB — if they don't match, return 403 immediately without a DB round-trip); rate-limit user profile reads per authenticated user; IP-level rate limiting at the reverse proxy for unauthenticated enumeration attempts.

DFD - Level 1 - RAWG API

RAWG API DFD



DFD - Level 1 - RAWG API

Game Search (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

RAWG REST API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Details (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Details Request (Data Flow)

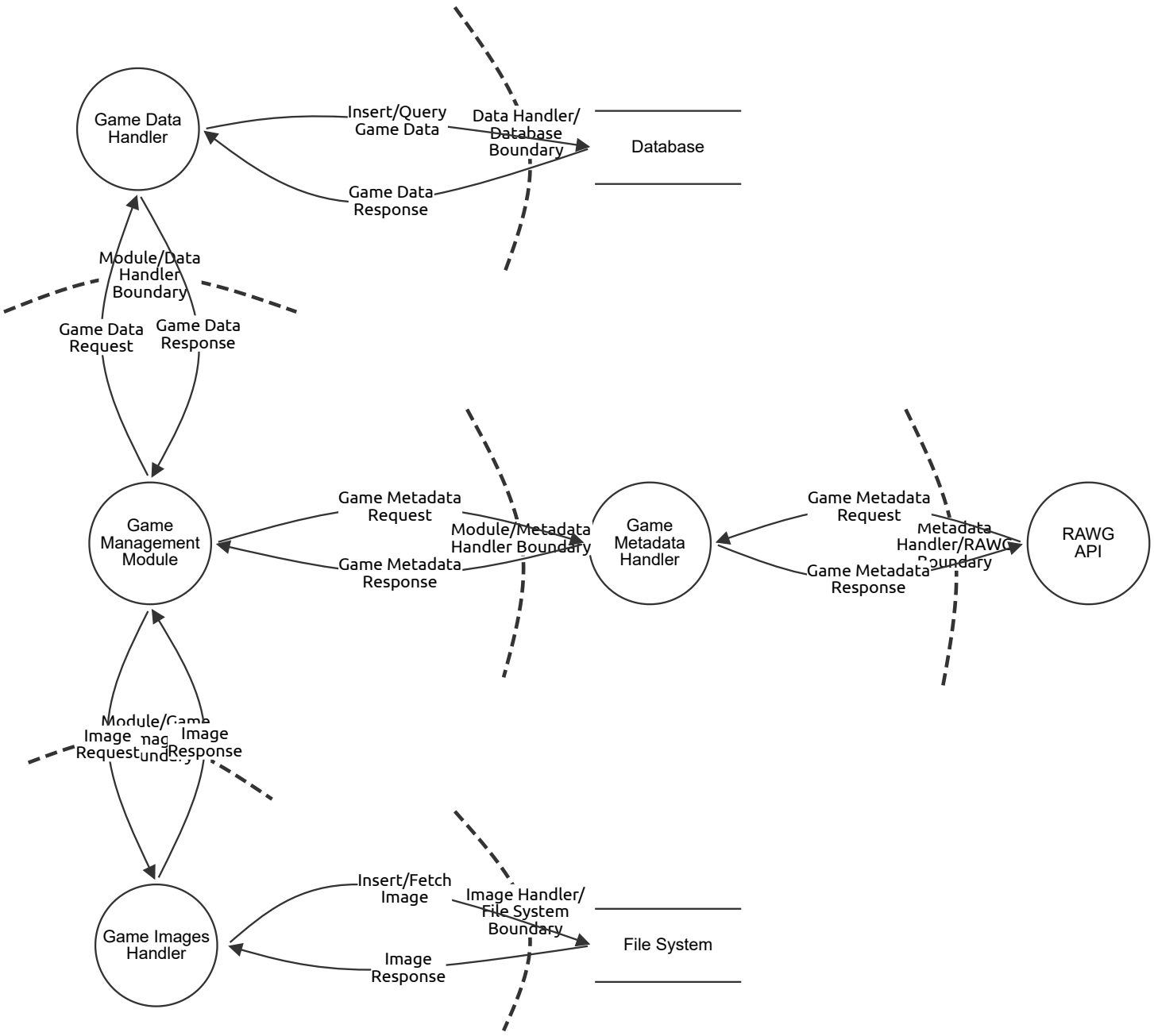
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Details Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

DFD - Level 2 - Game Management

ArcadeHaven DFD - Level 2 - Game Management



DFD - Level 2 - Game Management

Game Data Handler (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Database (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Management Module (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Handler (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

RAWG API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Images Handler (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

File System (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Data Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Data Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Insert/Query Game Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Game Data Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Image Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Image Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Insert/Fetch Image (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Image Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations